Graphs

Introduction to Graph Theory

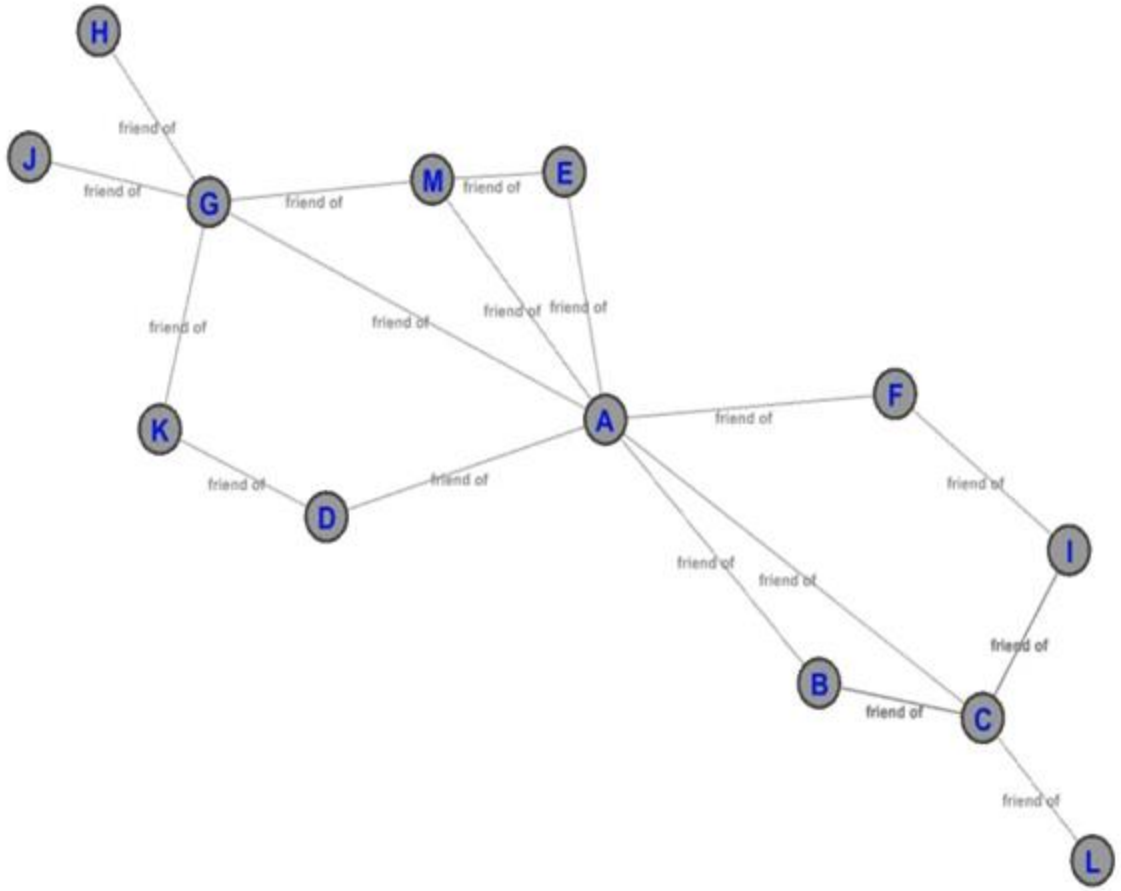
“Graph Theory”

Graph Processing

- What is Graph and its Applications
- Representing graphs- Storing a Graph - 3 ways
- Graph Featurization types
- Importance of vertices: Centrality measures
  - Common Measures:
    - Degree Centrality
    - Closeness Centrality
    - Betweenness Centrality

The **branch of data science** that deals with **extracting information** from graphs by performing analysis on them is known as **“Graph Analytics”.**

The advent of social networks, big data and e-commerce has re-emphasized the importance of analyzing “Graph” - a unique type of **data structure** - which depicts relationships among its entities.

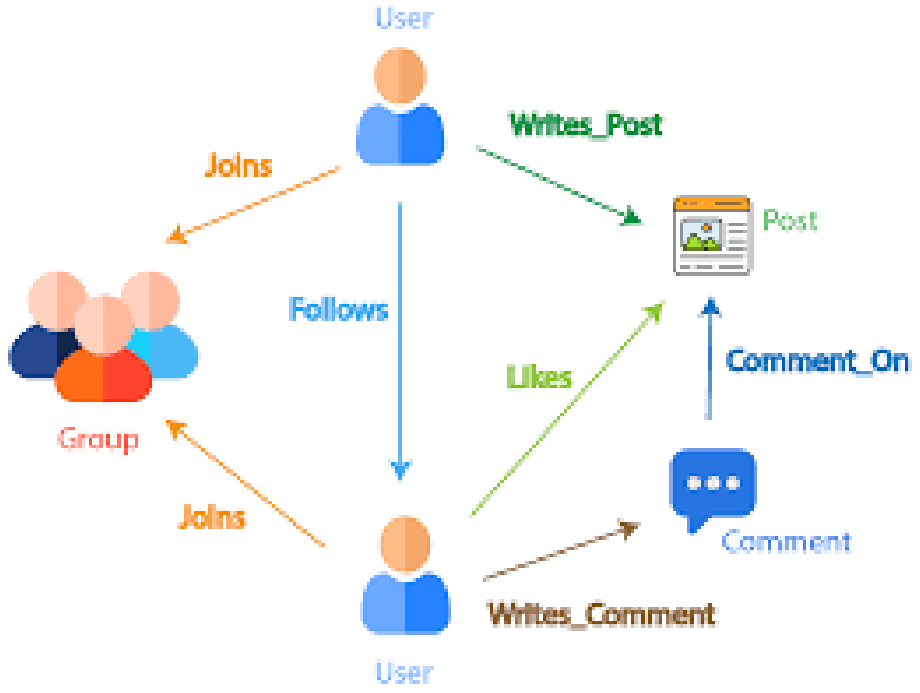


NETWORKS? GRAPHS?

Networks are systems of interrelated objects

Graphs are the mathematical models used to represent networks

In data science, we use algorithms on graphs to answer questions about real-world networks.



Graph Related Problems

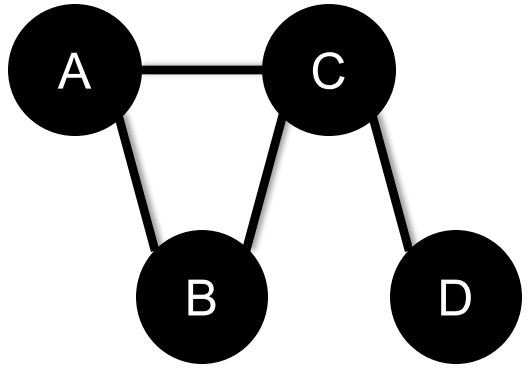
- Find the **shortest distance** between nodes
- **Predict** who will become friends if presented
- **Pages** you may **like**
- Many more

GRAPHS

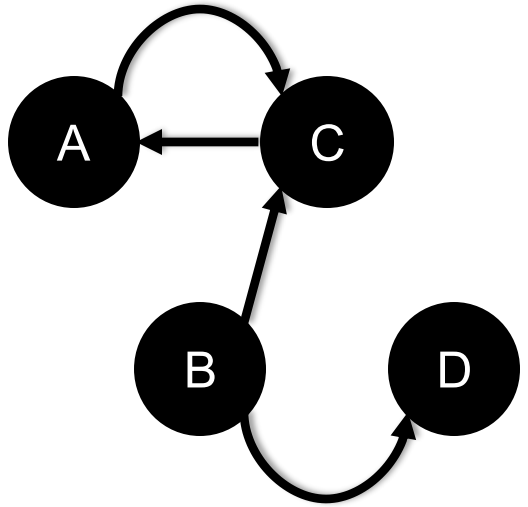
Nodes = Vertices
Edges = Arcs

A graph G = (V,E) is a set of vertices V and edges E

Edges can be undirected or directed



V = {*A, B, C, D*}
E = {*(A,B), (B,C), (C,D), (A,C)*}



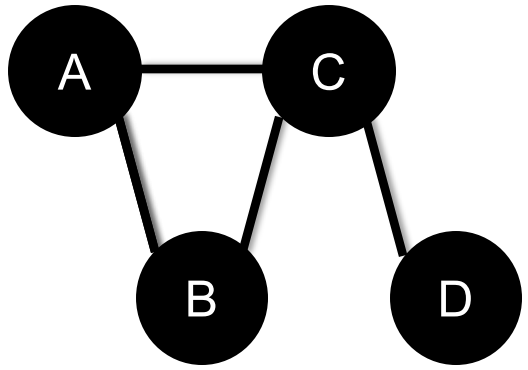
V = {*A, B, C, D*}
E = {*(A,C), (C,A), (B,C), (B,D)*}

Examples of directed vs undirected graphs

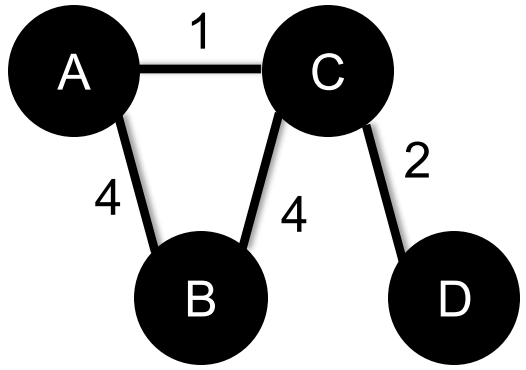
GRAPHS

Edges can be **unweighted** or **weighted**

- Unweighted : all edges have unit weight



Unweighted

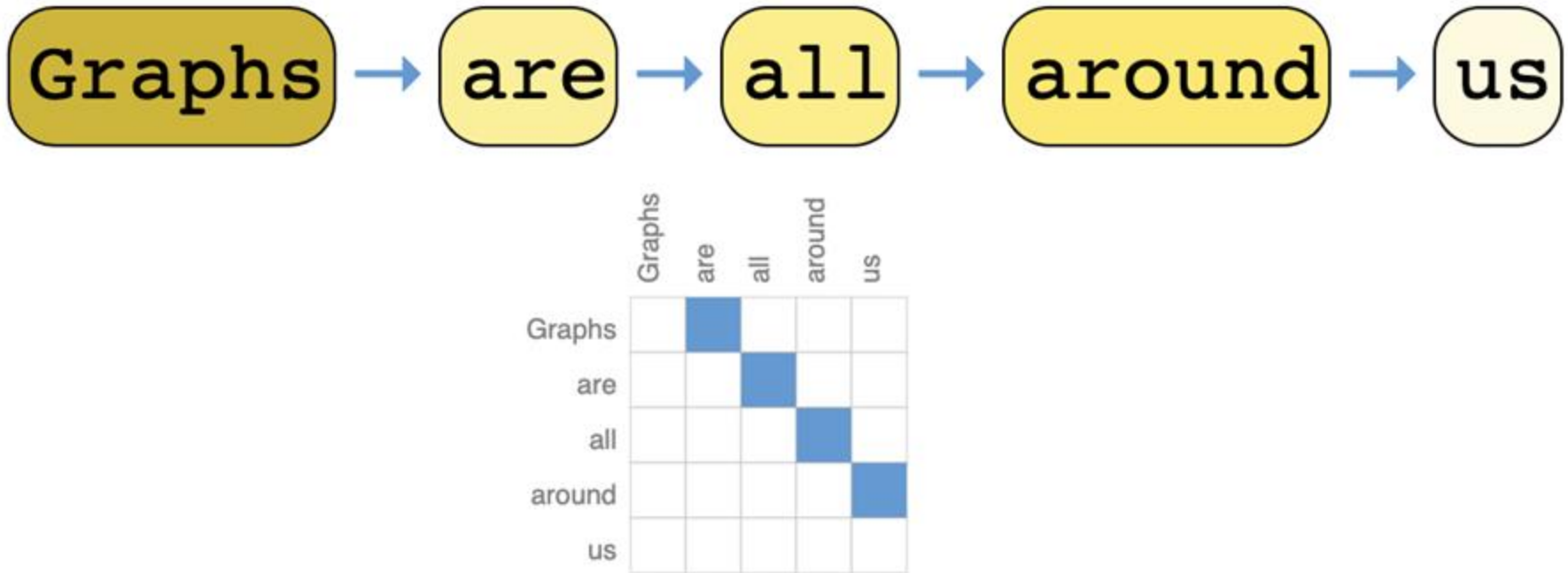


Weighted

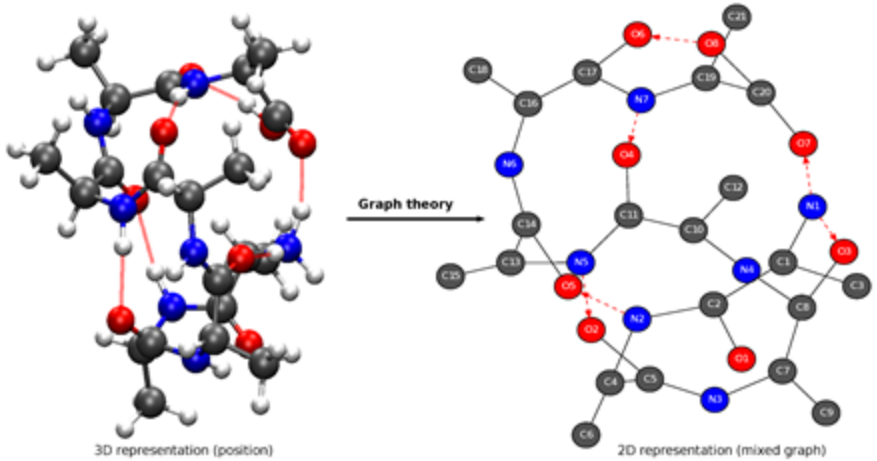
Examples of unweighted and weighted graphs

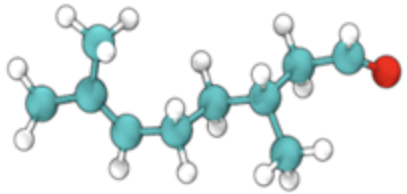
GRAPHS AND THE NETWORKS THEY REPRESENT

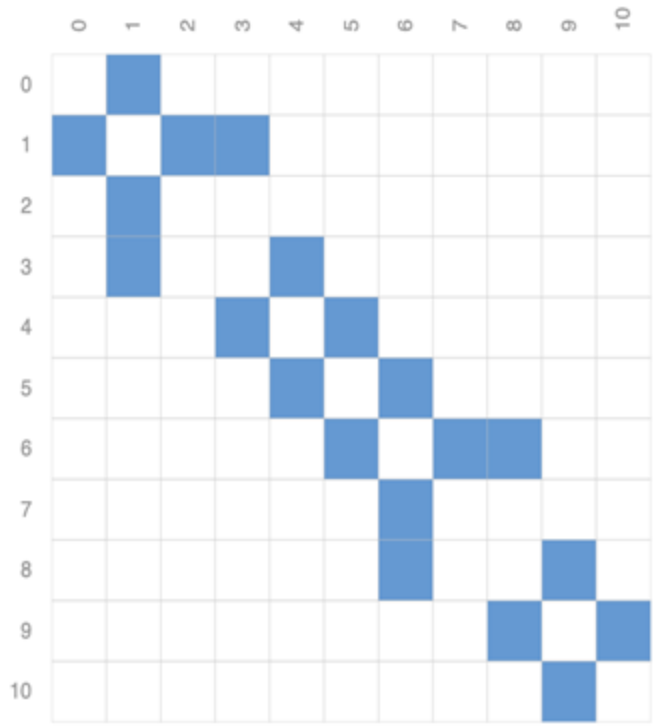
Text as graphs

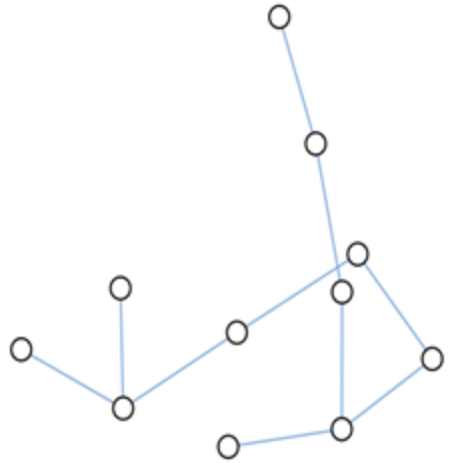


Molecular Structures as Graph





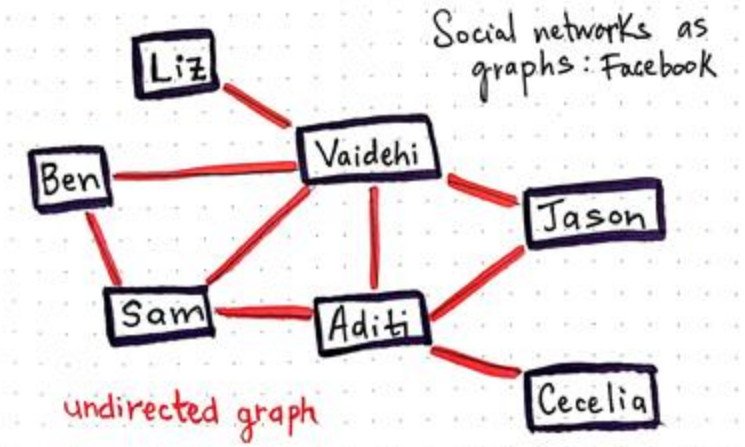


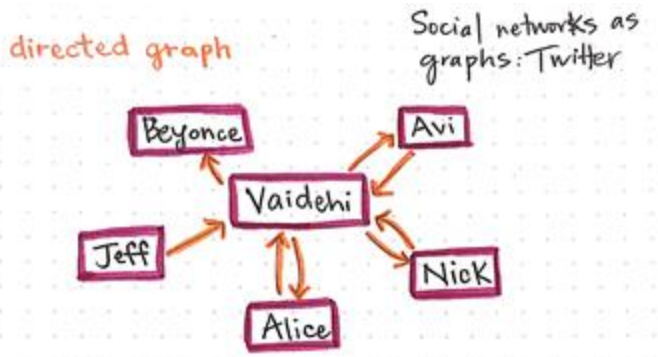


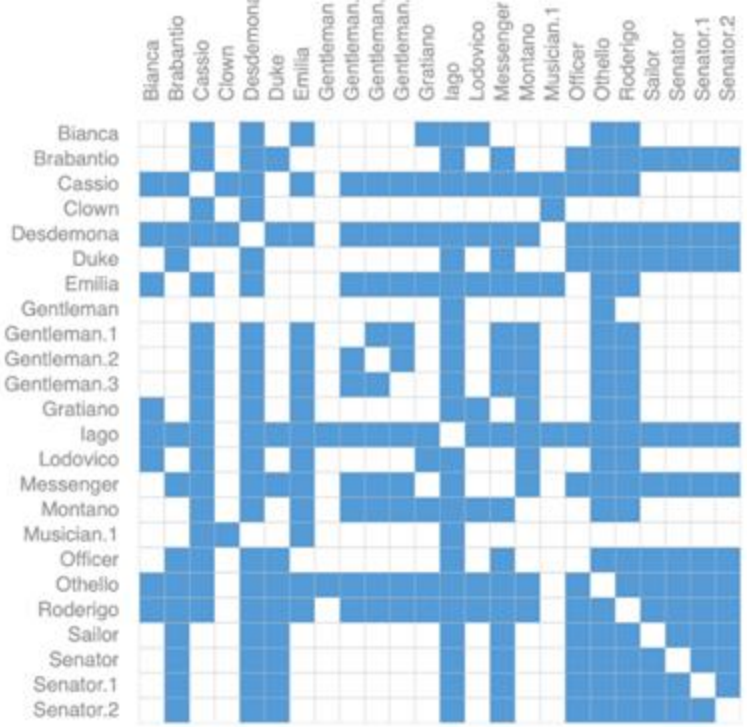
(Left) 3d representation of the Citronellal molecule
(Center) Adjacency matrix of the bonds in the molecule
(Right) Graph representation of the molecule.

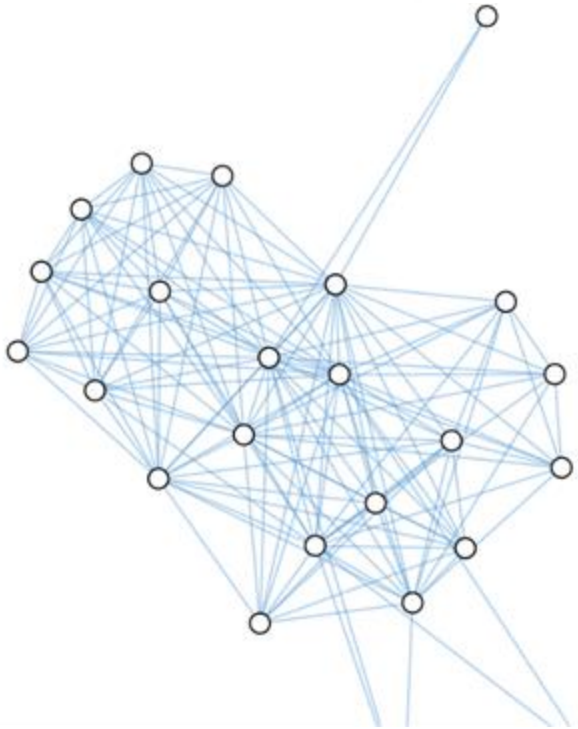
Social Networks as Graphs

A graph representing groups of people by modelling individuals as nodes, and their relationships as edges.









(Left) Image of a scene from the play “Othello”.
(Center) Adjacency matrix of the interaction between characters in the play.
(Right) Graph representation of these interactions.

IMAGES CAN BE VIEWED AS GRAPHS

A way of visualizing the connectivity of a graph is **through its adjacency matrix.**

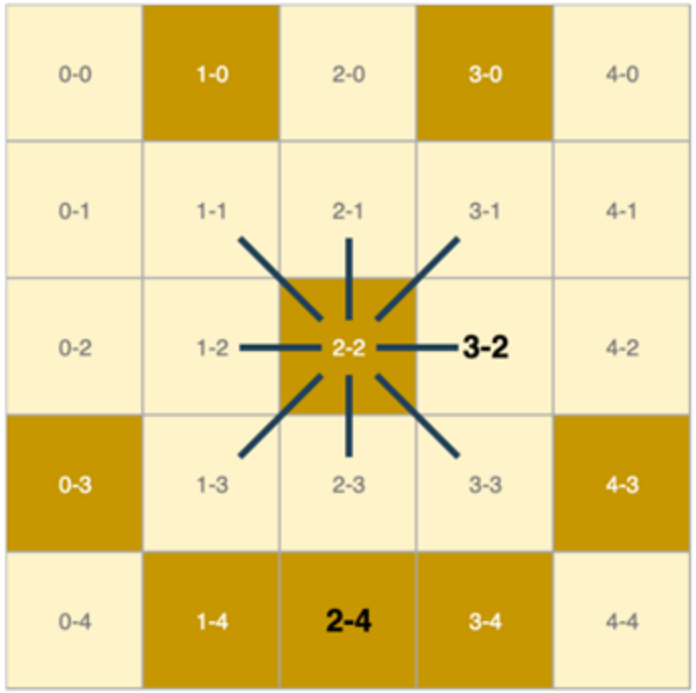
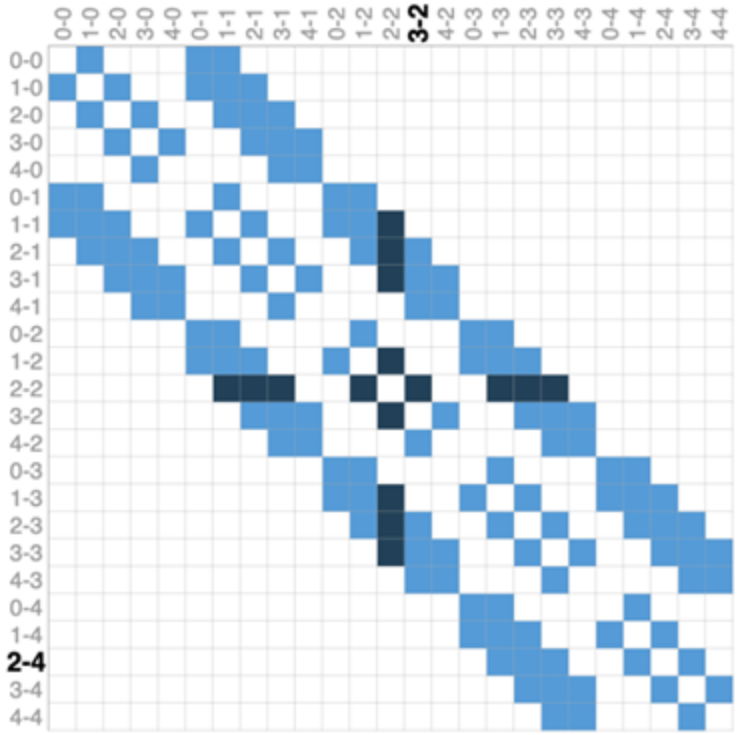
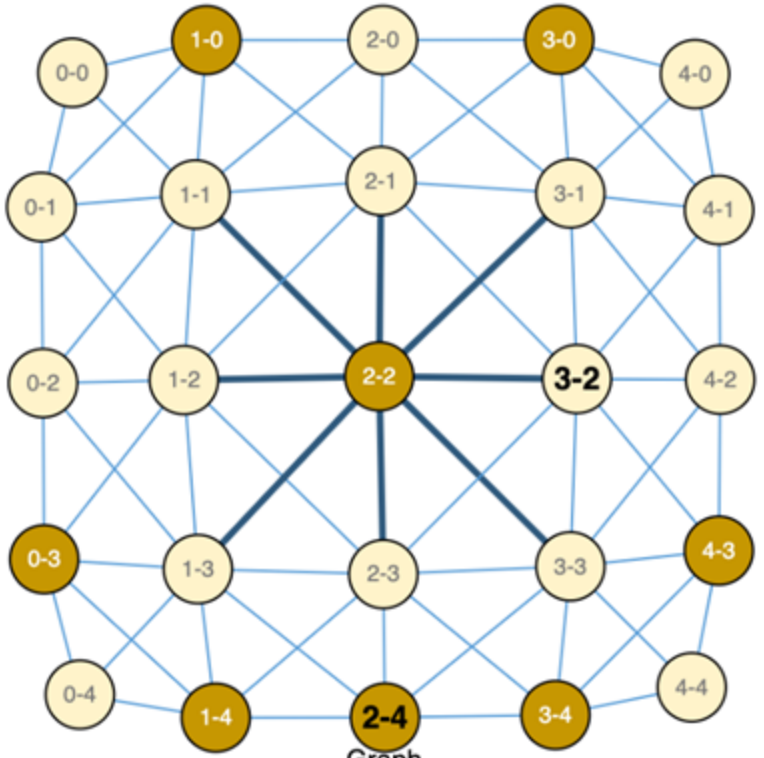


Image Pixels



Adjacency Matrix



Graph

NETWORKX

NetworkX is a Python library for storing, manipulating, and analyzing (small- and medium-sized) graphs

- Uses Matplotlib for final image
- https://networkx.github.io/
- conda install -c anaconda networkx

```
import networkx as nx

G=nx.Graph()
G.add_node(”class”)
G.add_edge(1,2)

print(list(G.nodes()))
print(list(G.edges()))
```

```
[1, 2, ‘class’]
[(1,2)]
```

STORING A GRAPH

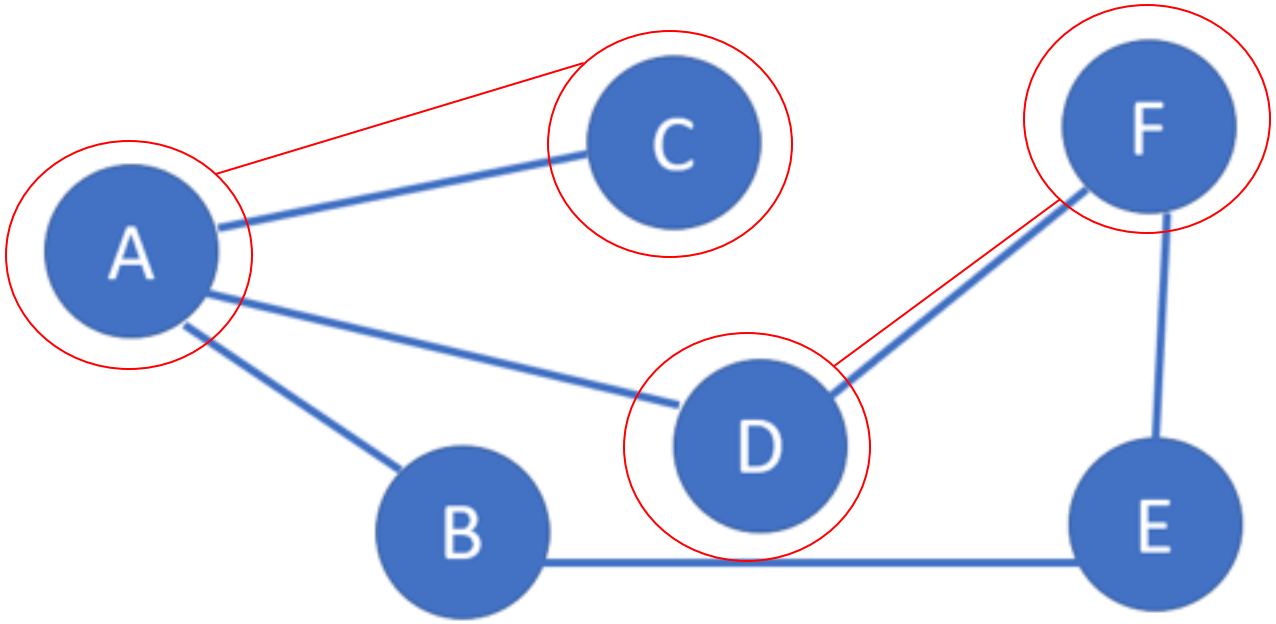
Three main ways to represent a graph in memory:

1. Adjacency lists
2. Adjacency dictionaries
3. Adjacency matrix

Graph Terms

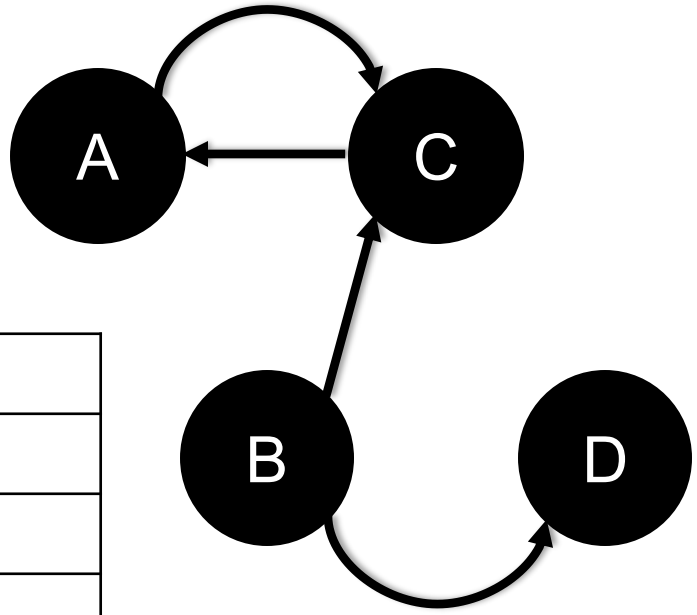
Adjacency: Two vertices are adjacent if they are *connected by an edge*.

E.g: A and C are adjacent, D and F are adjacent etc.



(1) ADJACENCY LISTS

For each vertex, store an array of the vertices it connects to store each vertex neighbors in a list



Vertex	Neighbors
A	[C]
B	[C, D]
C	[A]
D	[]

Pros: Simple to use for edge-specific operations:

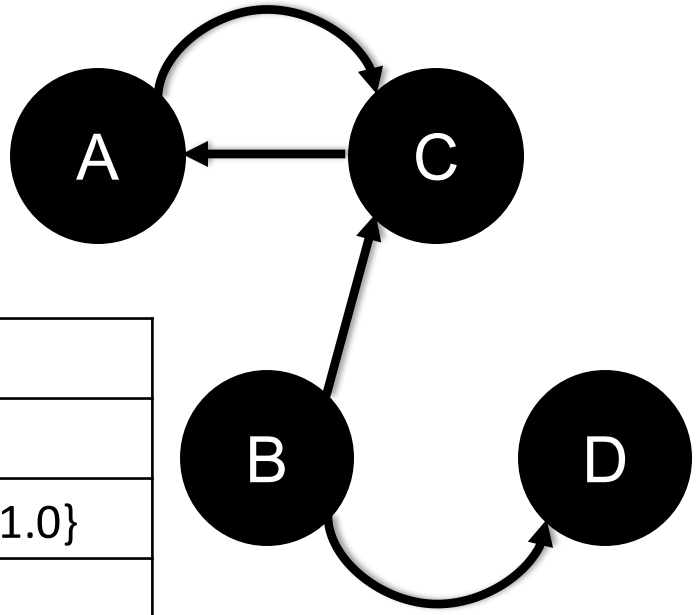
- Easy to iterate over all outgoing edges and the neighbors of any vertex.
- Easy to add an edge (simply append it)

Cons: Slow for vertex-specific operations:

- requiring list traversal, O(degree(u))) and
- being **less memory-efficient for dense graphs**

(2) ADJACENCY DICTIONARIES

For each vertex, store a dictionary of vertices it connects to Instead of lists, adjacency dictionaries use key-value pairs, where the key represents a vertex and the value is a list of its neighbors



Vertex	Neighbors
A	{C: 1.0}
B	{C: 1.0, D: 1.0}
C	{A: 1.0}
D	{}

Pros: O(1) to add, remove, query edges

- Accessing a specific key in a dictionary is fast and predictable.

Cons: Overhead (memory, caching, etc).

- Each vertex maintains a dictionary for its neighbors, potentially increasing memory usage.

(3) ADJACENCY MATRIX

Store the connectivity of the graph in a matrix represents connections between vertices in a matrix (2D array), where each cell indicates whether there's an edge between two vertices.

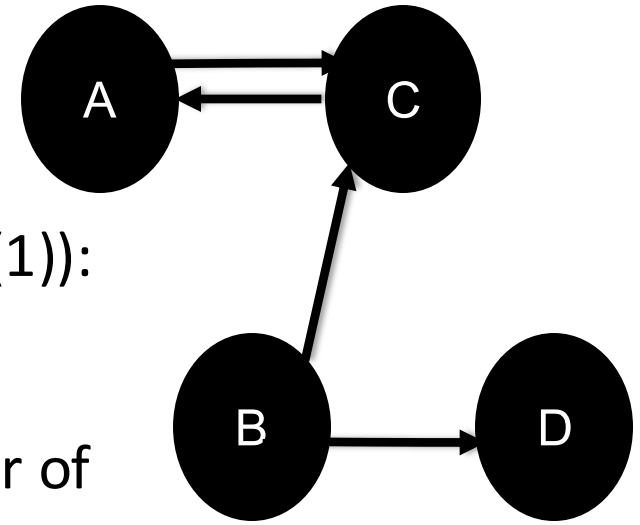
Pros:

Easy to implement, Fast Edge Queries (O(1)):

Cons:

- $O(|V|^2)$ space regardless of the number of edges

Almost always stored as a sparse matrix

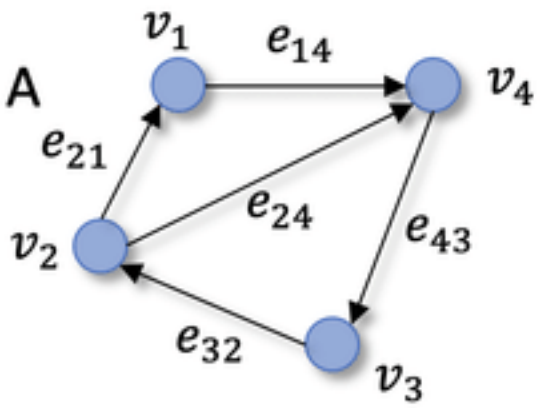


To \ From	A	B	C	D
A	0	0	1	0
B	0	0	0	0
C	1	1	0	0
D	0	1	0	0

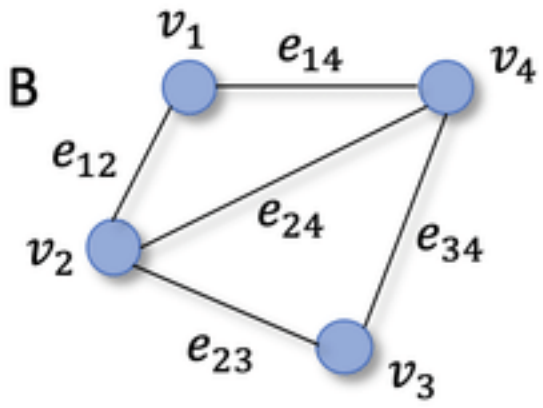
[i][j] represents the presence (or weight) of an edge between vertex i and vertex j.

(3) ADJACENCY MATRIX

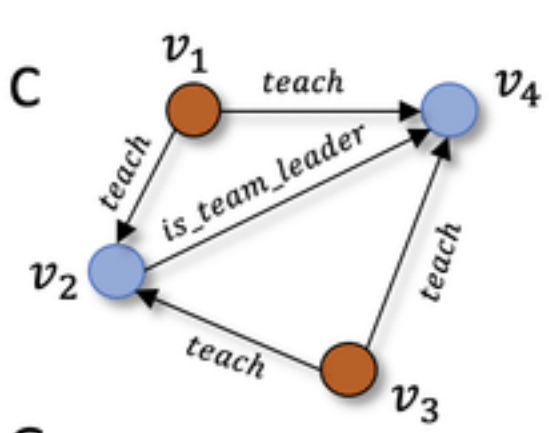
Directed graph G(V,E)



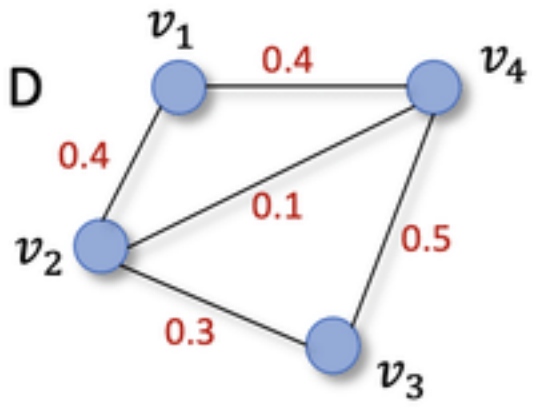
Undirected graph G(V,E)



Knowledge graph G(V,E)



Weighted graph G(V,E)



E

	v_1	v_2	v_3	v_4
v_1	0	0	0	1
v_2	1	0	0	1
v_3	0	1	0	0
v_4	0	0	1	0

F

	v_1	v_2	v_3	v_4
v_1	0	1	0	1
v_2	1	0	1	1
v_3	0	1	0	1
v_4	1	1	1	0

G

	v_1	v_2	v_3	v_4
v_1	0	1	0	1
v_2	0	0	0	1
v_3	0	1	0	1
v_4	0	0	0	0

H

	v_1	v_2	v_3	v_4
v_1	0	0.4	0	0.4
v_2	0.4	0	0.3	0.1
v_3	0	0.3	0	0.5
v_4	0.4	0.1	0.5	0

[i][j] represents the presence (or weight) of an edge between vertex i and vertex j.

Choosing the Right Graph Representation: Adjacency List, Dictionary, and Matrix

Sparse Graphs (few edges compared to vertices):

- **Adjacency List:** Best for memory efficiency and quick neighbor iteration.
- **Adjacency Dictionary:** Great for fast edge lookups and when you need to store extra data on edges (e.g., edge weights).

Dense Graphs (many edges compared to vertices):

- **Adjacency Matrix:** Best choice, as it allows quick edge lookups and is space-efficient when the graph is dense.

Graphs with Many Vertex and Edge Modifications (dynamic graphs):

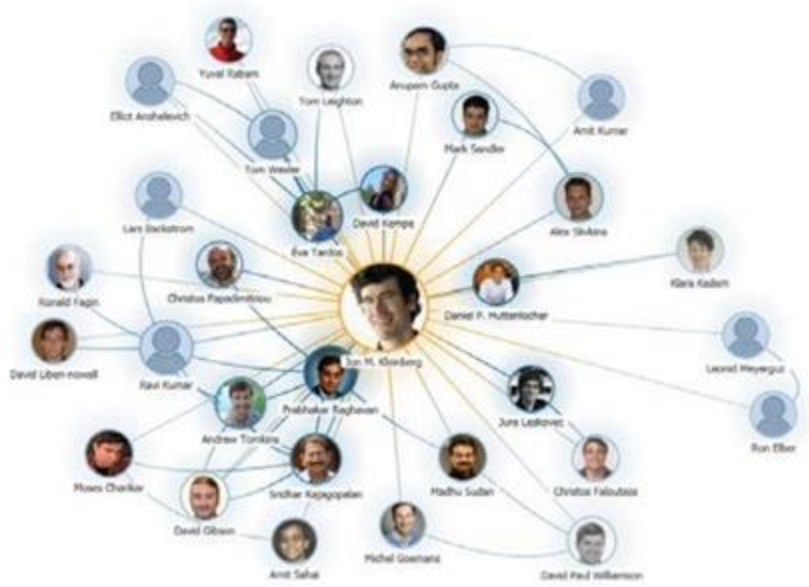
- **Adjacency List:** Easy to update, especially for adding or removing edges.
- **Adjacency Dictionary:** Good for fast edge modifications and adding attributes to edges.

Graph Traversal (BFS, DFS, Dijkstra's Algorithm, etc.):

- **Adjacency List:** Typically the best choice for performance in traversal operations, especially with sparse graphs.

Graph Featurization

By transforming graph data into suitable numerical features or representations, machine learning models can be applied to various tasks.



	A	B	C	D	E	F
A	0	1	1	1	0	0
B	1	0	0	0	1	1
C	1	0	0	0	0	1
D	1	0	0	0	0	0
E	0	1	0	0	0	0
F	0	1	1	0	0	0

The most natural way to do this for a graph is to use something like the **adjacency matrix**, but each graph can have many different ones!

Graph Featurization

Graph featurization transforms graph structural information into numerical features that can be used for tasks such as classification, clustering, or link prediction.

1. **Graph Level Features:** features describe the entire graph as a whole and can capture global structural patterns (such as graph diameter, average path length, modularity, graph centralization etc.).

1. **Edge Level Features:** features describe the relationships or connections between two nodes (Edge Betweenness Centrality, Common Neighbors etc) .

1. **Node Level Features:** features describe individual nodes in the graph.
    a. Degree (In and Out)
    b. Centrality Measure

THE VALUE (IMPORTANCE) OF A VERTEX

Centrality: Represents a “measure of importance”

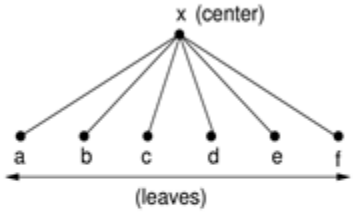
A very important concept in identifying important nodes in a graph in Graph Analytics.

Centrality measures how "central" or important a node is within a graph.

- Usually for nodes.
- Some measures can also be defined for edges (or subgraphs, in general).

There are various types of centrality metrics, each providing a different perspective on a node's importance in the network.

Example:



- The **center** node is “structurally more important” than the other nodes.

Reasons for the importance of the center node:

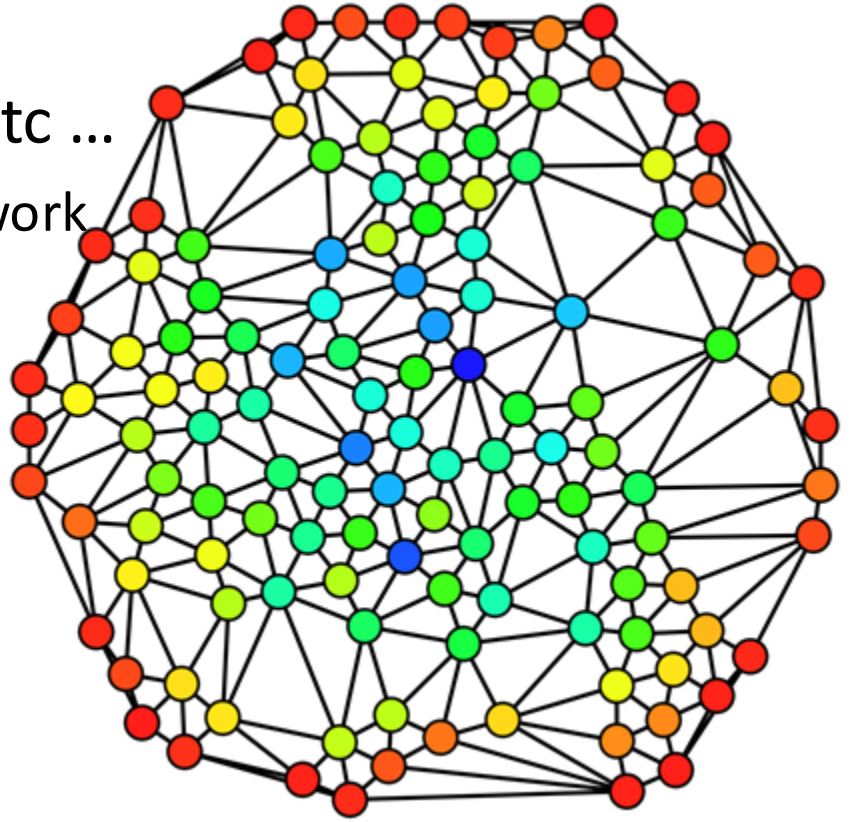
- The center node has the maximum possible degree.
- It lies on the shortest path (“geodesic”) between any pair of other nodes (leaves).

IMPORTANCE OF VERTICES

Not all vertices are equally important

Centrality Analysis:

- Find out the most important node(s) in one network
- Used as a feature in classification, for visualization, etc …
  - highlighting key nodes based on their significance in the network



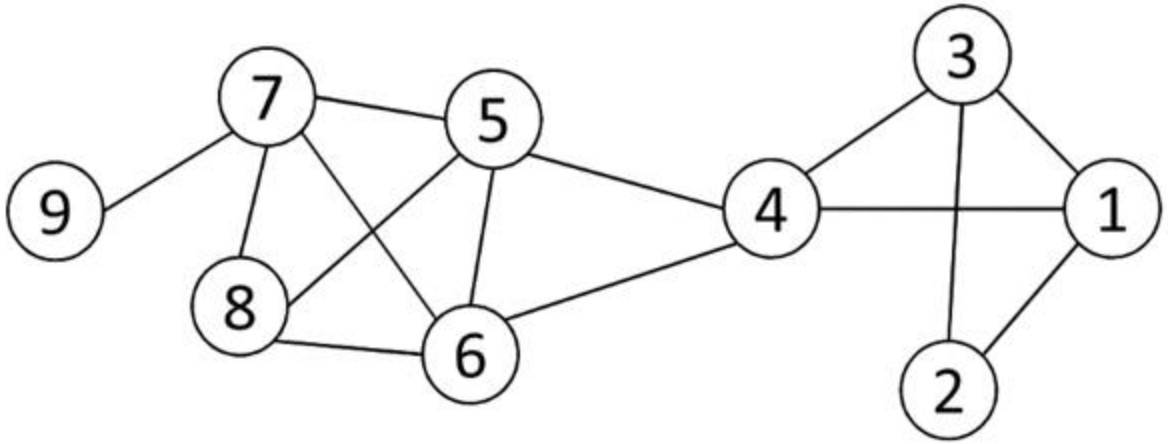
Commonly-used Measures

1. Degree Centrality
2. Closeness Centrality
3. Betweenness Centrality

1. DEGREE CENTRALITY

1. DEGREE CENTRALITY

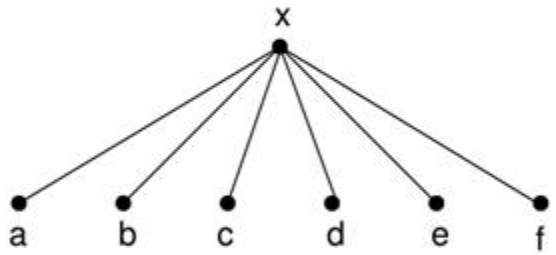
Degree Centrality: defines the importance of a node based on the degree of that node. The higher the degree, the more crucial it becomes in the graph.



- Quantifies the number of direct connections a node has in a graph.
- Used to find the individual nodes' "popularity", by looking into the number of incoming and outgoing relationships.

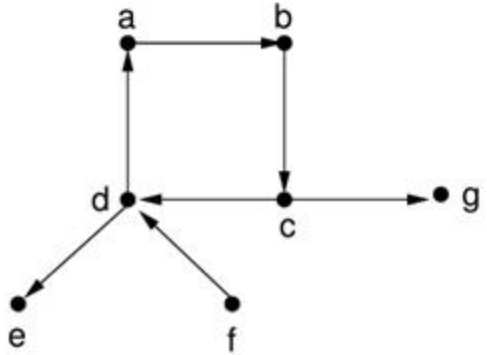
- For an **undirected** graph, the **degree** of a node is the number of edges incident on that node.
- For a **directed** graph, both **indegree** (i.e., the number of incoming edges) and **outdegree** (i.e., the number of outgoing edges) must be considered.

Example 1:



- Degree of $x = 6$.
- For all other nodes, degree $= 1$.

Example 2:



- Indegree of $b = 1$.
- Outdegree of $d = 2$.

Total Degree Centrality(v)=
In-degree(v)+Out-degree(v)

1. DEGREE CENTRALITY: EQUATIONS

Degree Centrality:

$$C_D(v_i) = d_i = \sum_j A_{ij}$$

d_i = Number of edges connected to node v_i

Here, A_{ij} represents the elements of the adjacency matrix, where $A_{ij} = 1$ if there is an edge between nodes i and j, and $A_{ij} = 0$ otherwise.

$$C'_D(v_i) = d_i/(n-1)$$

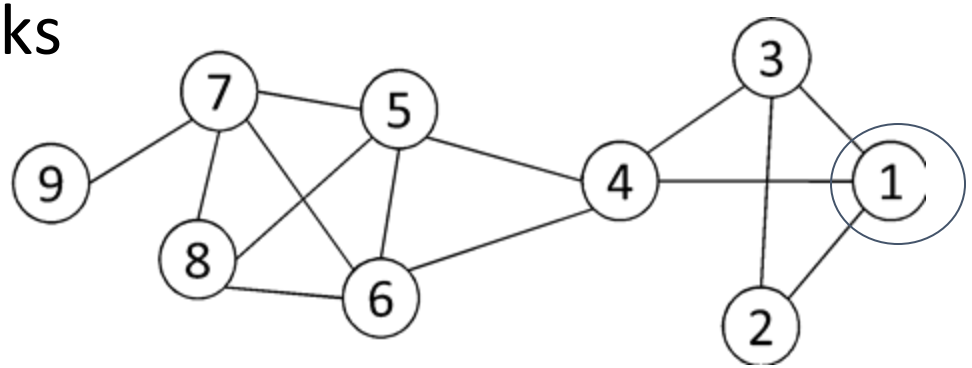
Normalized Degree Centrality:

n = the number of nodes in the network

1. DEGREE CENTRALITY

The importance of a vertex is determined by the number of vertices adjacent to it

- The larger the degree, the more important the vertex is
- Only a small number of vertex have high degrees in many real-life networks



For Vertex 1,

- **degree centrality** is 3;
- **Normalized degree centrality** is 3/(9-1)=3/8.

Example: Consider a social network where each node represents a person, and edges represent friendships. **Nodes with higher degree centrality are those with more friends (such as popular individuals in social network)**.

2. CLOSENESS CENTRALITY

2. CLOSENESS CENTRALITY

“Central” vertices are important;

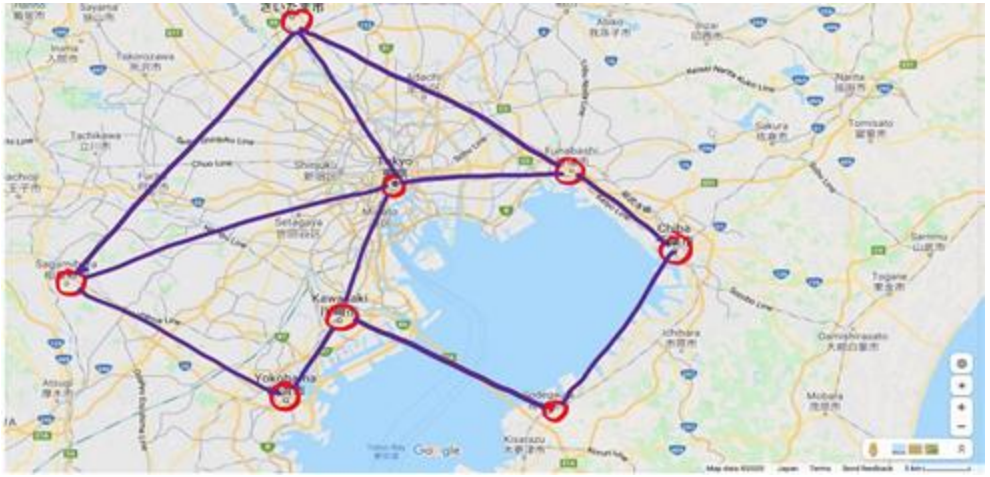
Closeness centrality identifies a node's importance based on how close it is to all the other nodes in the graph.

- The closeness is also known as geodesic distance (GD).

Geodesic Distance or shortest path distance

Geodesic distance is the shortest path between two nodes in a network, measured in terms of the number of edges traversed.

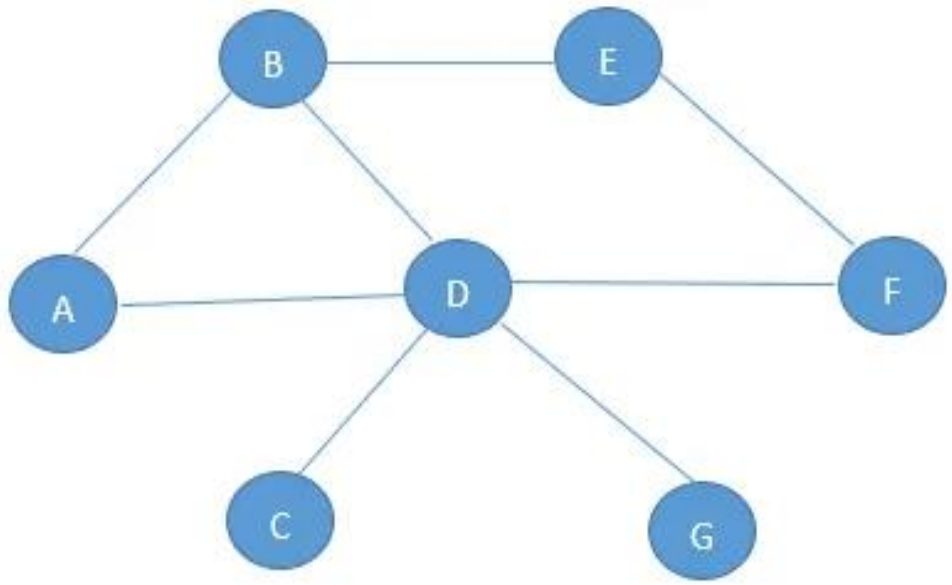
- It represents the minimum number of edges that must be traversed to go from one node to another in a graph.



To measure “closeness centrality” , we calculate the average of the shortest path length from “the node” to every other node in the network/graph.

Geodesic Distance

The Geodesic distance d between two nodes a and b is defined as the number of edges/links between these two nodes on the shortest path (path with minimum number of edges) between them.



d(a, b) No. of edges between a and b on the shortest path from a to b, if a path exists from a to b

d(a, b) = 0 if a=b

d(a, b) = ∞ (Infinity), if no path exists from a to b

The geodesic distance between A and F:
We can reach F from A by going through B and E or by going through D. However, the shortest path from F to A is through D(2 edges), hence the geodesic distance d(A,F) will be defined as 2 as there are 2 edges between A and F.

2. CLOSENESS CENTRALITY: Example

Emphasizing nodes that can quickly interact with the rest of the network as they can reach the whole network more quickly than non-central vertices (Celebrities ideal for marketing or an influencing strategy).

A high closeness centrality means that the node can reach all other nodes quickly, i.e., it is near to many nodes. This is an indicator of being a central or influential node in the network

- A low average shortest distance indicates higher centrality, suggesting the node is well-connected and central to the network.

2. CLOSENESS CENTRALITY: Equations

Importance measured by how close (central) a vertex is to other vertices.

Normalized Closeness Centrality: $C_C(v_i) = \left[\frac{1}{n-1}\sum_{j\neq i}^{n} g(v_i, v_j)\right]^{-1} = \frac{n-1}{\sum_{j\neq i}^{n} g(v_i, v_j)}$

g(vi,vj) = the distance (*length of the shortest path*) between vertices vi and vj

$$Closeness\ Centrality\ Score(u) = \frac{number\ of\ nodes - 1}{\sum\ (distance\ from\ u\ to\ all\ other\ nodes)}$$

Steps to Calculate CLOSENESS CENTRALITY

1. **Calculate the shortest path distance** from the node v to all other nodes in the graph.
    - For each pair of nodes v and u, find the shortest path $d(v, u)$ (this can be done using algorithms like **Dijkstra's Algorithm** for weighted graphs or **Breadth-First Search (BFS)** for unweighted graphs).
2. **Sum the shortest distances** from node v to all other nodes u.
3. Apply the formula to calculate closeness centrality using either the unnormalized or normalized version of the formula.

$$Closeness\ Centrality\ Score(u) = \frac{number\ of\ nodes - 1}{\sum\ (distance\ from\ u\ to\ all\ other\ nodes)}$$

2. CLOSENESS CENTRALITY

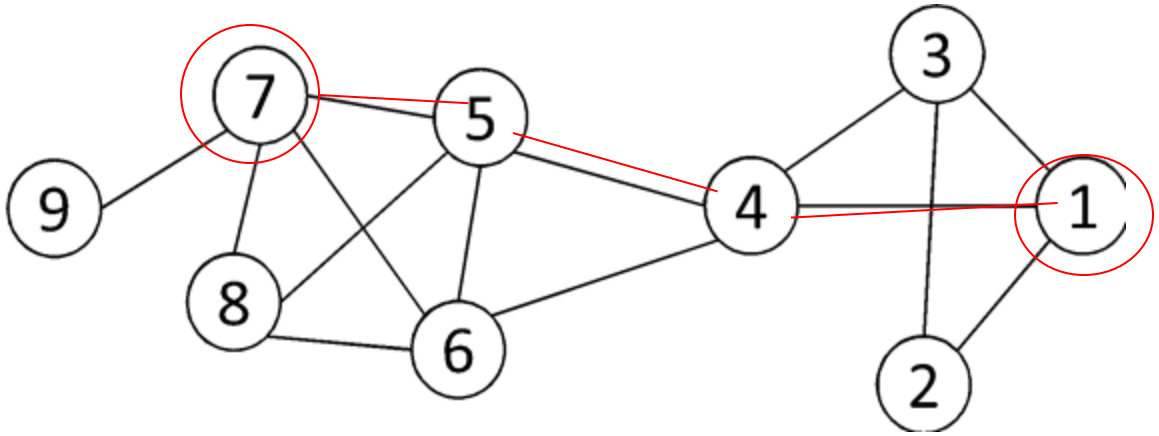


Table shows the **shortest path from each one of the nodes to all others**.

Table 2.1: Pairwise geodesic distance

Node	1	2	3	4	5	6	7	8	9
1	0	1	1	1	2	2	3	3	4
2	1	0	1	2	3	3	4	4	5
3	1	1	0	1	2	2	3	3	4
4	1	2	1	0	1	1	2	2	3
5	2	3	2	1	0	1	1	1	2
6	2	3	2	1	1	0	1	1	2
7	3	4	3	2	1	1	0	1	1
8	3	4	3	2	1	1	1	0	2
9	4	5	4	3	2	2	1	2	0

Step 01

Step 02:

$$C_C(3) = \frac{9-1}{1+1+1+2+2+3+3+4} = 8/17 = 0.47,$$

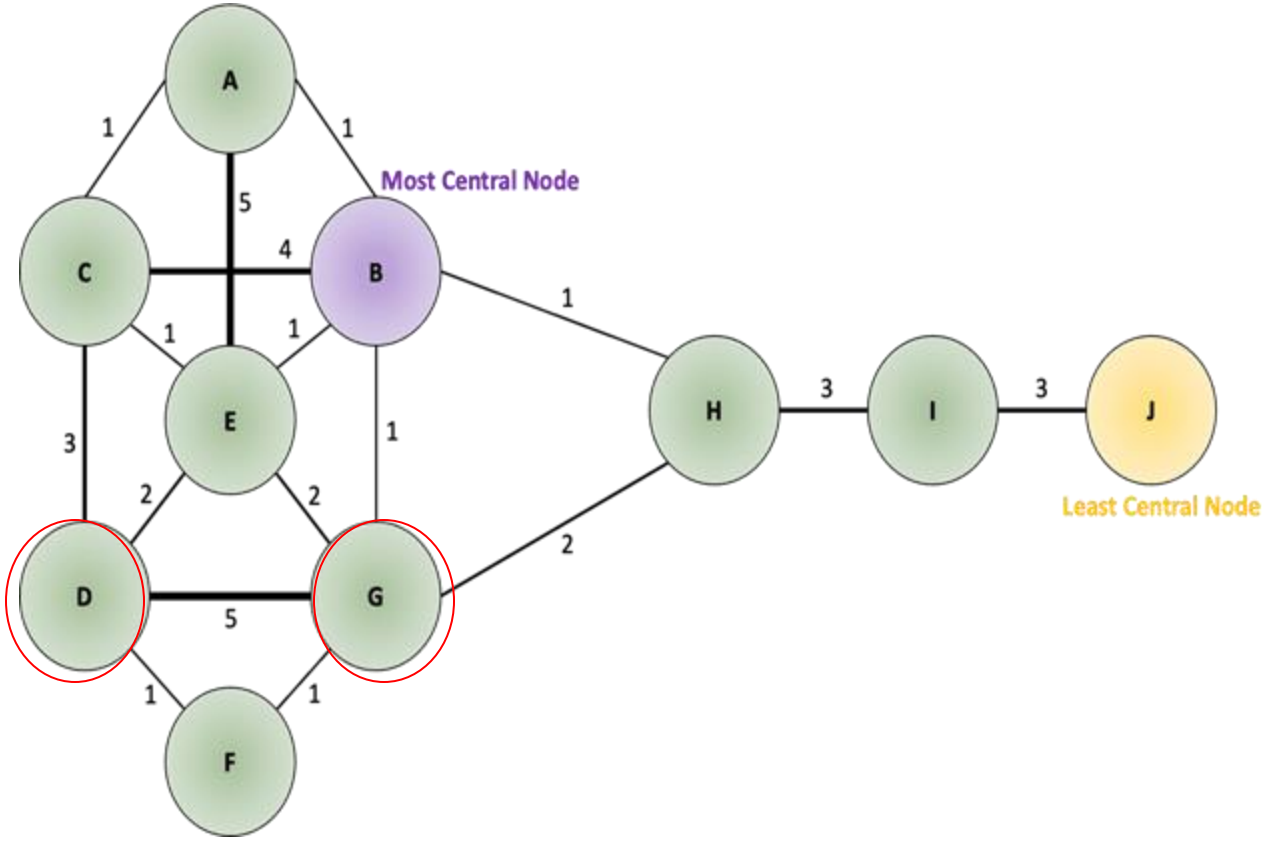
$$C_C(4) = \frac{9-1}{1+2+1+1+1+2+2+3} = 8/13 = 0.62.$$

Vertex 4 is more central than vertex 3

Closeness Centrality: For Weighted Network

In this case, the shortest path is not the one with the least hops but the one with the **minimum total weighted distance.**

For instance, in the example shown in Figure, accounting for weighted paths, the shortest distance between nodes D and G includes two hopes (D-F-G=2), instead of the single hop option (D-G=5)



	A	B	C	D	E	F	G	H	I	J	Normalized Score
A	0	1	1	4	2	3	2	2	5	8	0.32
B	1	0	2	3	1	2	1	1	4	7	**0.41**
C	1	2	0	3	1	4	3	3	6	9	0.28
D	4	3	3	0	2	1	2	4	7	10	0.25
E	2	1	1	2	0	3	2	2	5	8	0.35
F	3	2	4	1	3	0	1	3	6	9	0.28
G	2	1	3	2	2	1	0	2	5	8	0.35
H	2	1	3	4	2	3	2	0	3	6	0.35
I	5	4	6	7	5	6	5	3	0	3	0.2
J	8	7	9	10	8	9	8	6	3	0	0.13

Table 2: Geodesic Distance Matrix, resulting from the nodes in the graph network of Figure 2.

3. BETWEENNESS CENTRALITY

Degree Centrality vs Closeness Centrality vs Betweenness centrality

"You know everyone" - Degree Centrality:

If a node has a high degree centrality, it means that it is **directly connected to a large number of other nodes** in the network. In social network terms, this could be interpreted as "knowing" many people directly.

"You know a bunch of people who know everyone" - Closeness Centrality:

Closeness centrality measures how quickly a node can reach all other nodes in the network. If a node has high closeness centrality, it is, on average, close to all other nodes. In social network terms, it means that you (or the node) are well-connected to others either directly or through a short path.

Degree Centrality vs Closeness Centrality vs Betweenness centrality

If you know a bunch of people who act as bridges between different groups in the network, you might have high betweenness centrality.

- Play a crucial role in connecting different parts of the network, even if don't have the highest number of direct connections (high degree).

Measures the importance of a node in a network based upon **how many times it occurs in the shortest path between all pairs of nodes (finding bridge nodes)** in a graph.

For example, in a telecommunications network, **a node with higher betweenness centrality would have more control over the network**, because more information will pass through that node

3. BETWEENNESS CENTRALITY

- **Vertex betweenness (traditional):** measures the importance of nodes (vertices) in a graph. It indicates how many shortest paths between other nodes pass through a particular node.

- **Edge Betweenness:** measures the importance of edges in a network, rather than nodes. It calculates how many shortest paths between pairs of nodes pass through a particular edge. This can help identify critical connections (edges) that hold parts of the network together.

3. BETWEENNESS CENTRALITY: EQUATION

Vertex betweenness counts the number of shortest paths that pass through one vertex

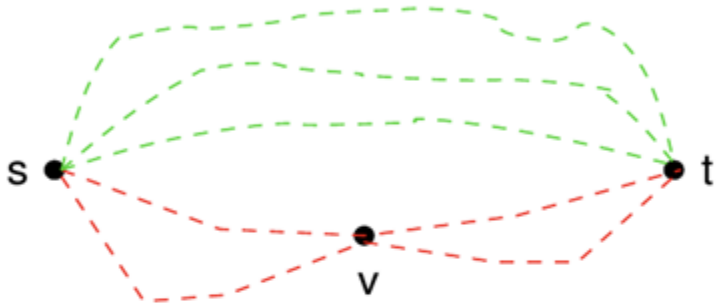
Betweenness Centrality: $$C_B(v_i) = \sum_{v_s \neq v_i \neq v_t \in V, s<t} \frac{\sigma_{st}(v_i)}{\sigma_{st}}$$

σ_{st} : The number of shortest paths between s and t

$\sigma_{st}(v_i)$: The number of shortest paths between s and t that pass v_i

3. BETWEENNESS CENTRALITY

Notation: Any shortest path between nodes s and t will be called an s-t **shortest path**.



- Let σ_{st} denote the number of all s-t shortest paths.
- Let $\sigma_{st}(v)$ denote the number of all s-t shortest paths that pass through node v.

Consider the ratio $\dfrac{\sigma_{st}(v)}{\sigma_{st}}$:

- This gives the fraction of s-t shortest paths passing through v.
- The larger the ratio, the more important v is with respect to the pair of nodes s and t.
- To properly measure the importance of a node v, we need to consider all pairs of nodes (not involving v).

Graphs

Introduction to Graph Theory

The Girvan-Newman algorithm

GIRVAN-NEWMAN ALGORITHM

The Girvan-Newman algorithm for the detection and analysis of community structure relies on the iterative elimination of edges that have the highest number of shortest paths between nodes passing through them.

By removing edges from the graph one-by-one, the network **breaks down into smaller pieces, so-called communities**.

The algorithm was introduced by Michelle Girvan and Mark Newman.

- What is “Community”
- How to remove/ eliminate edges that have the highest number of shortest paths - “Edge Betweenness”

COMMUNITY

Real-life graphs are not random

- E.g., in a social network people pick their friends based on their common interests and activities.

We expect that the nodes in a graph will be organized in communities.

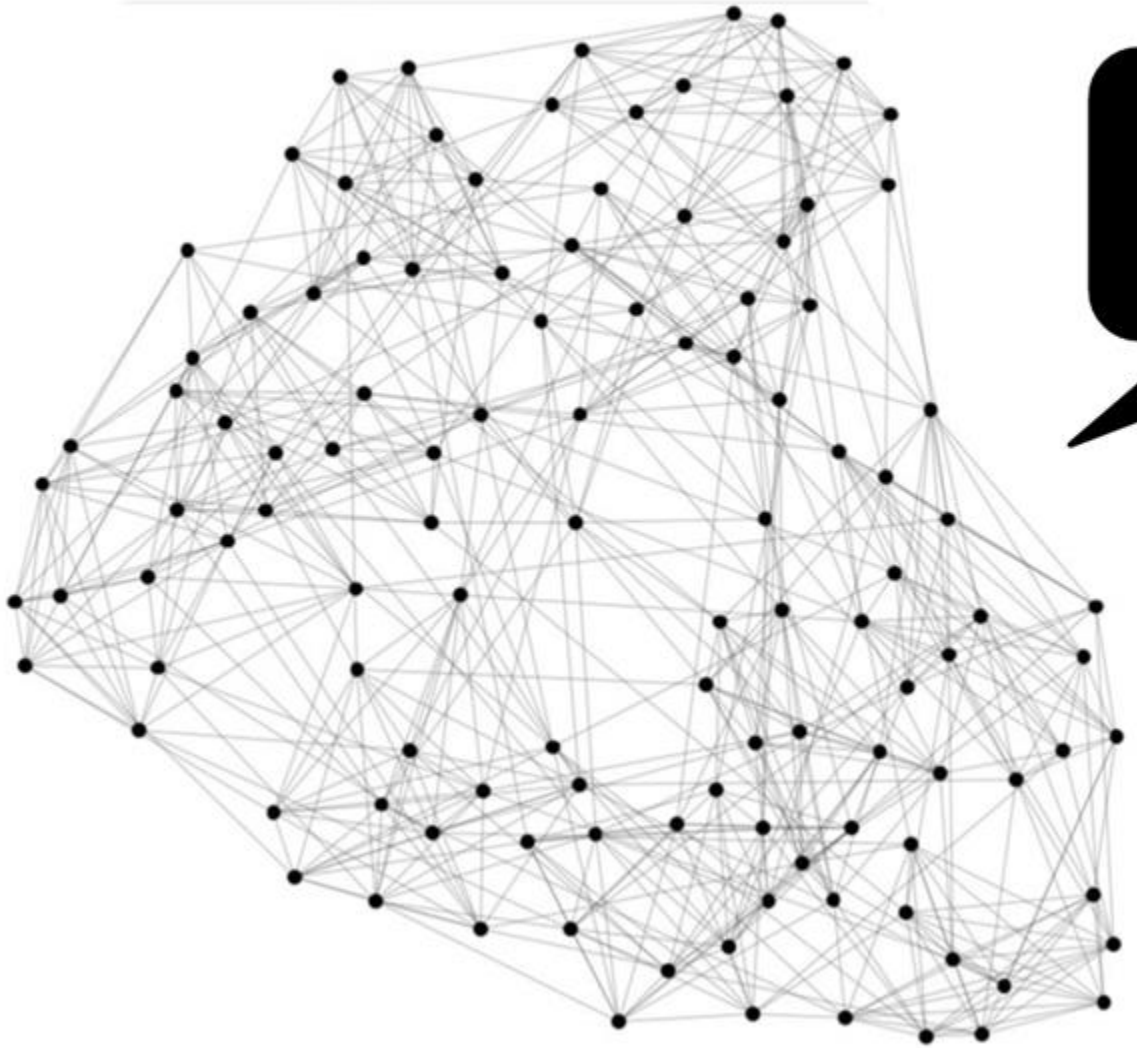
- Groups of vertices which probably share common properties and/or play similar roles within the graph

How do we find them?

- A subset of nodes in communities will be densely connected to each other, and sparsely (loosely) connected to the nodes in other communities in the same graph.

NCAA Football network

PROBLEM: CAN WE IDENTIFY GROUPS OF DENSELY CONNECTED NODES?

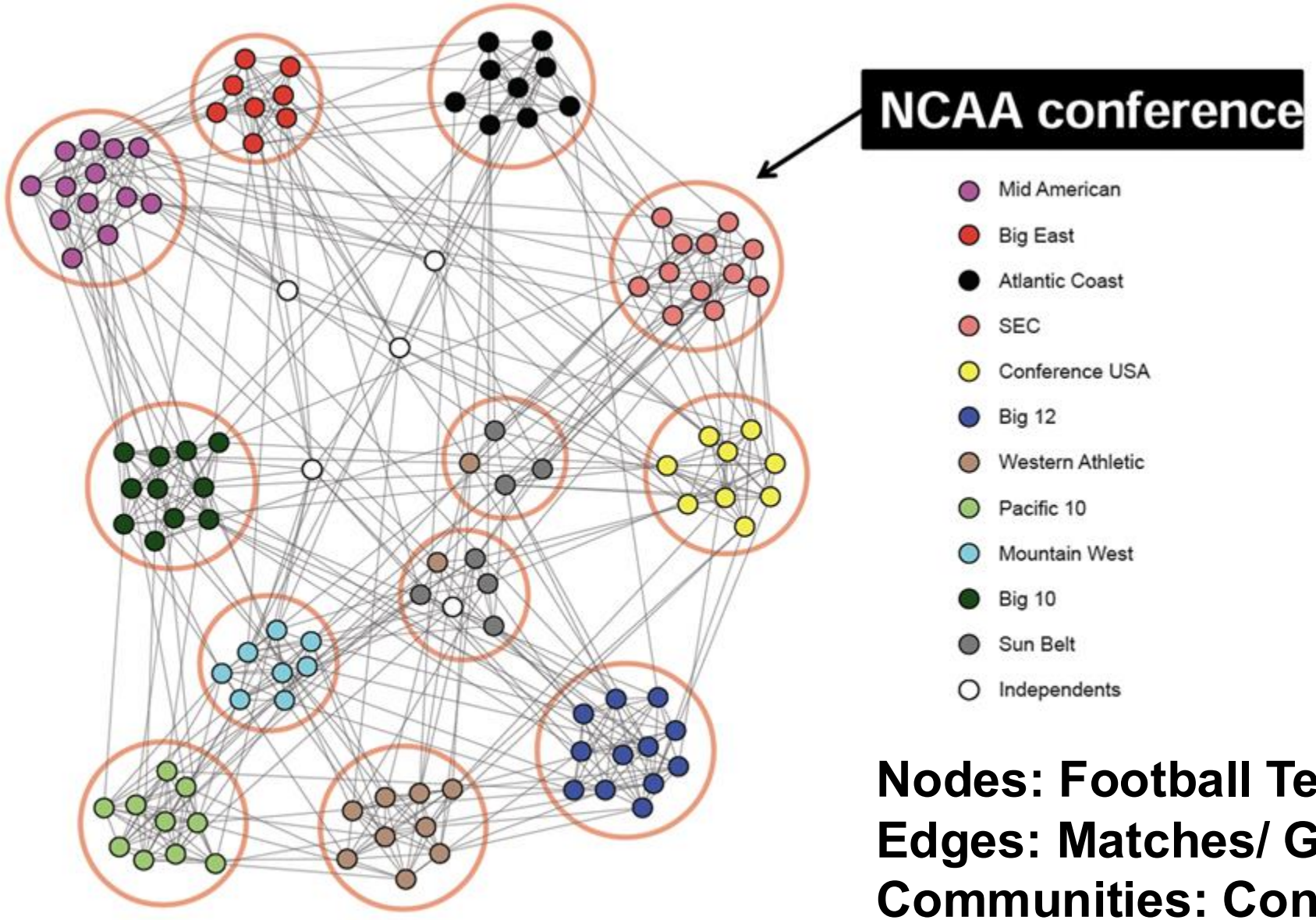


Can we identify node groups? (communities, modules, clusters)

Nodes: Football Teams
Edges: Games played

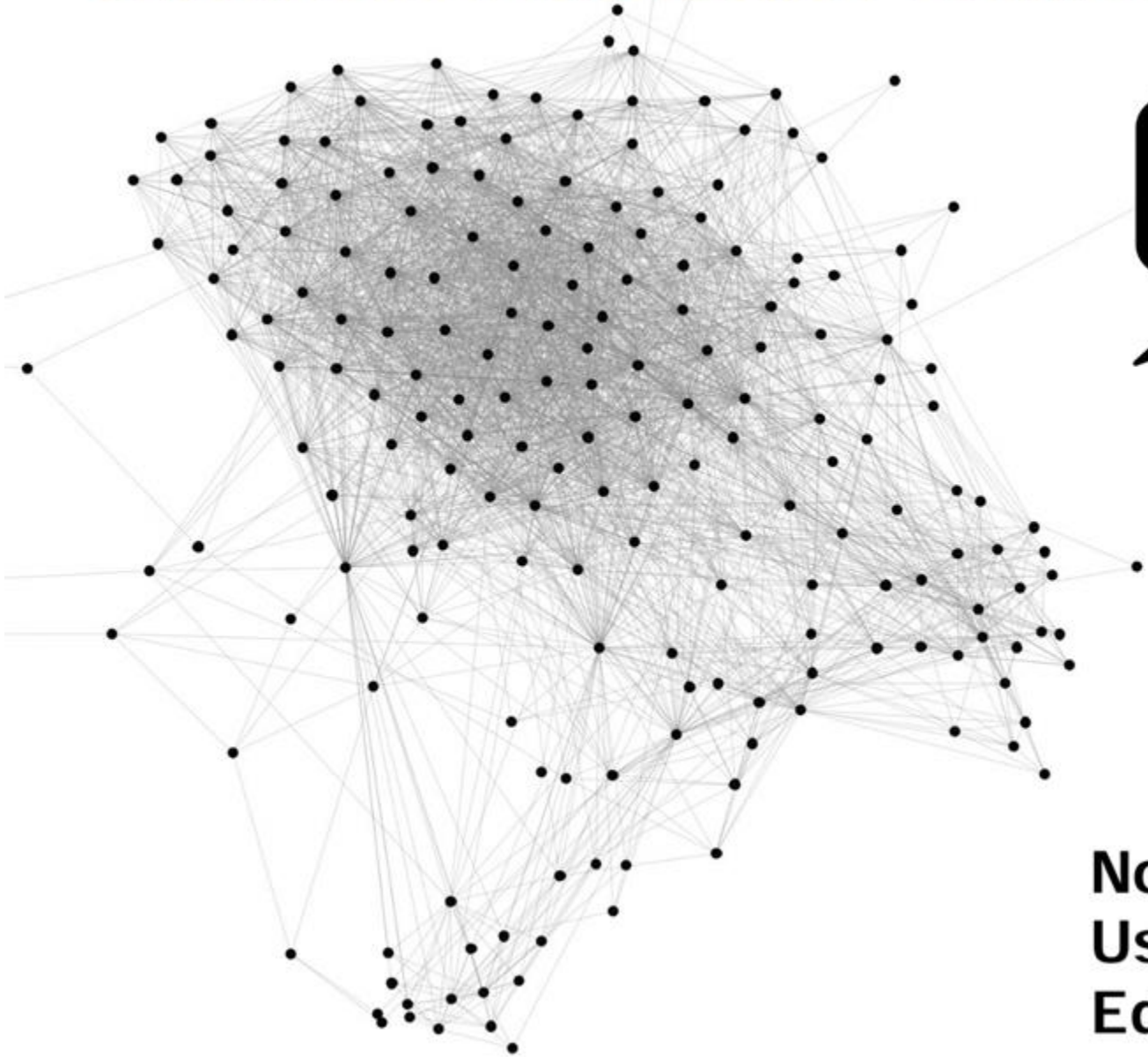
(Adapted from: Mining of Massive Datasets, http://www.mmds.org

COMMUNITIES: FOOTBALL CONFERENCES



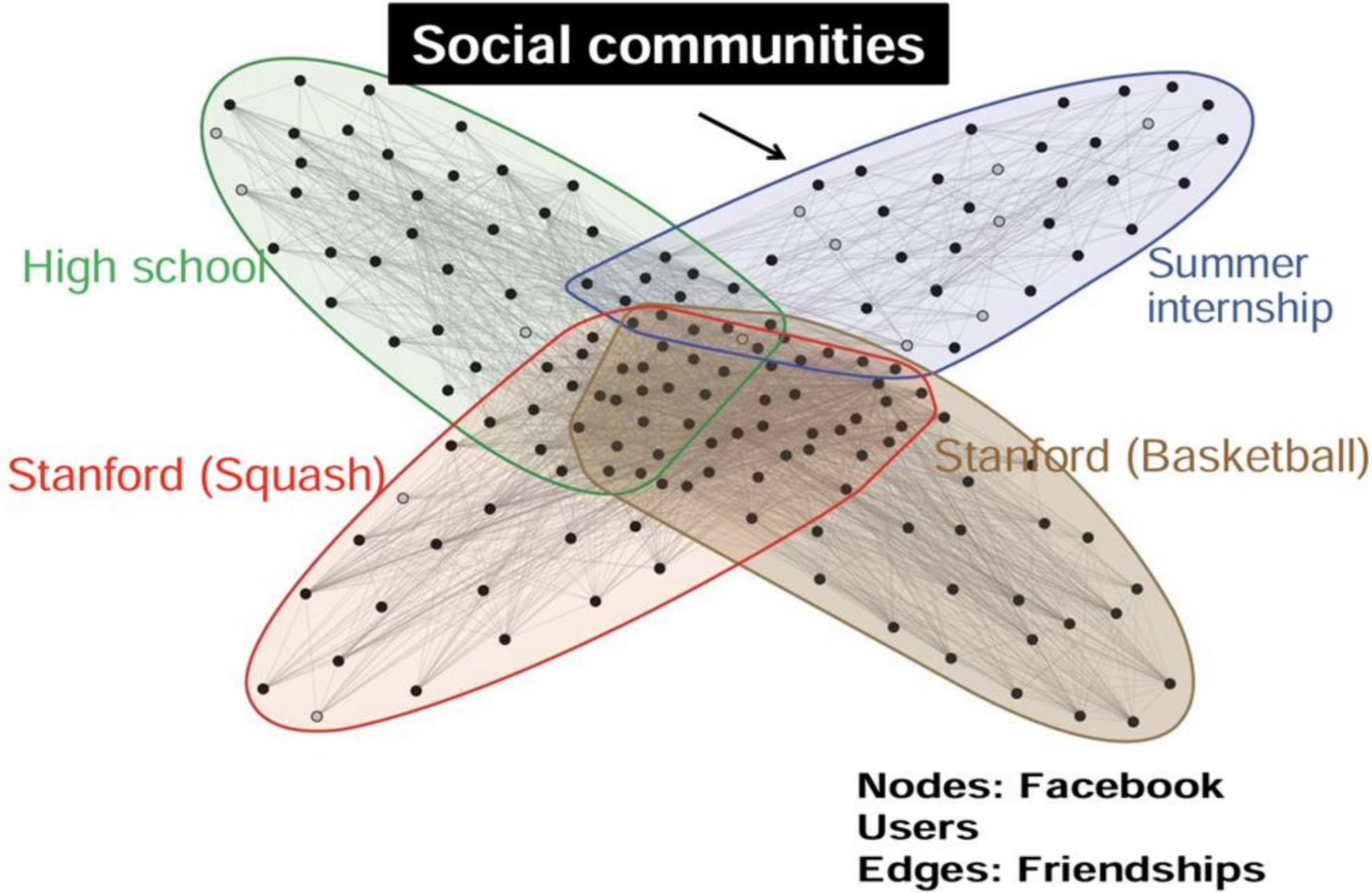
Nodes: Football Teams,
Edges: Matches/ Game played,
Communities: Conferences

Stanford Facebook network

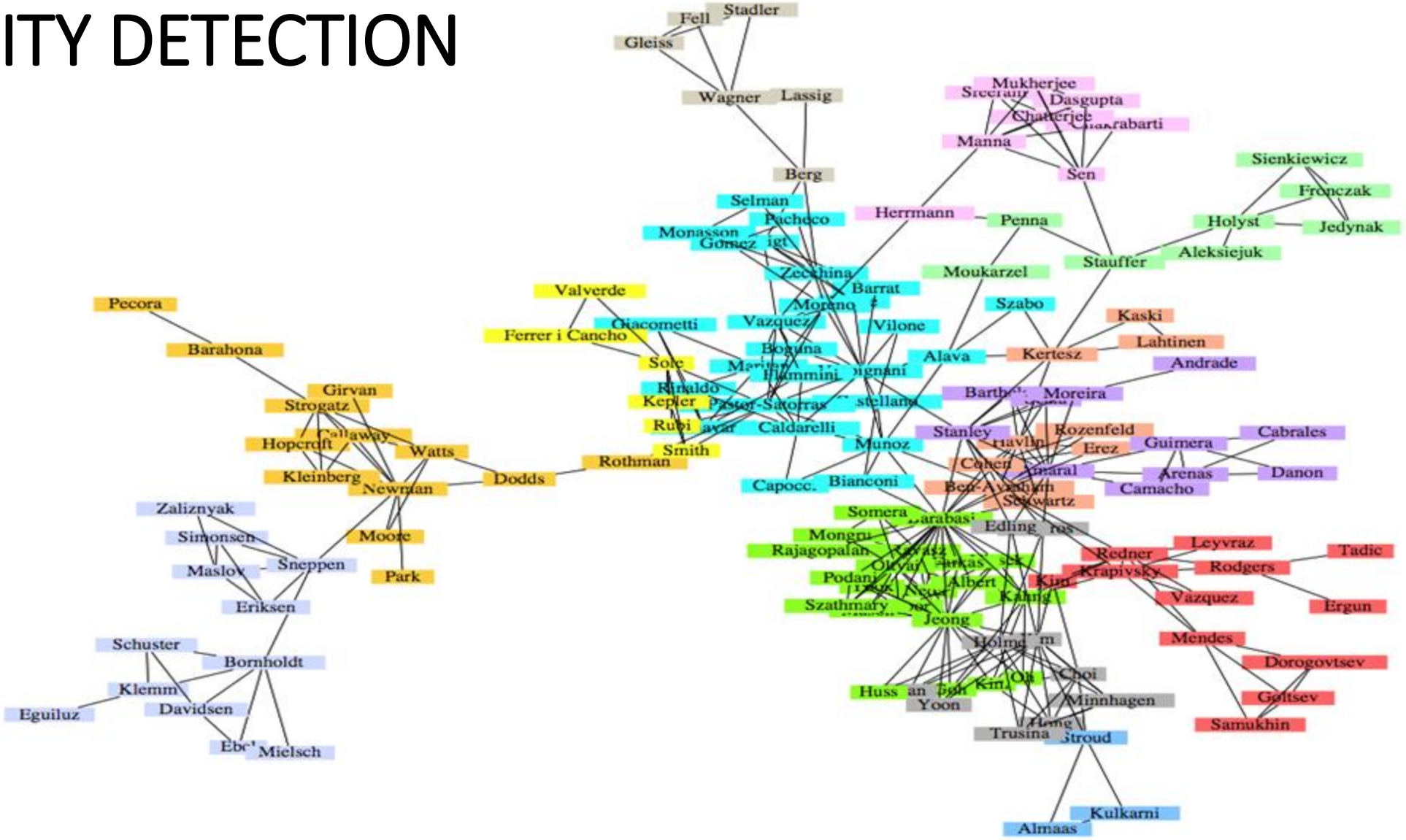


Can we identify social communities?

Nodes: Facebook Users
Edges: Friendships


Social communities
High school
Summer internship
Stanford (Squash)
Stanford (Basketball)
Nodes: Facebook Users
Edges: Friendships

COMMUNITY DETECTION

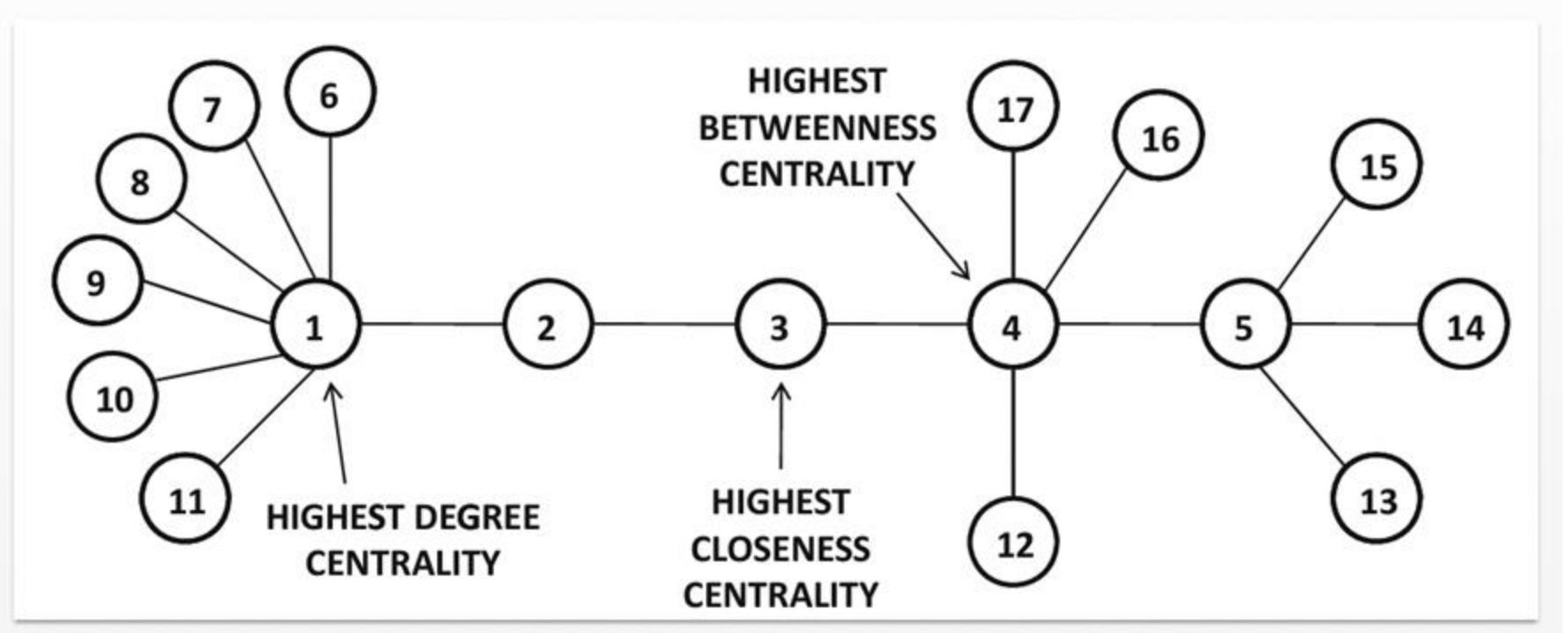


How can the tightly clustered groups be identified?

A co-authorship network of physicists and mathematicians

(Courtesy: Easley & Kleinberg)

RECAP: CENTRALITY

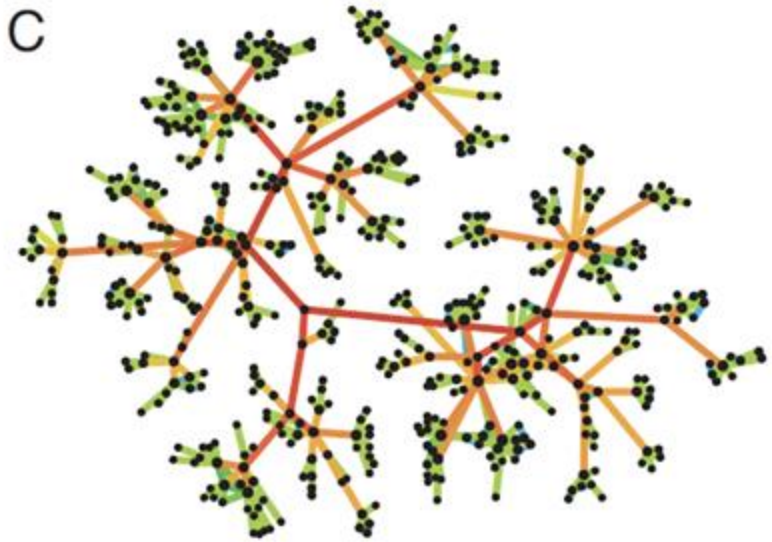


- **Betweenness:** Number of shortest paths through a node or a edge
- **Closeness:** Average distance to other nodes
- **Degree:** Number of connections to other nodes

EDGE IMPORTANCE

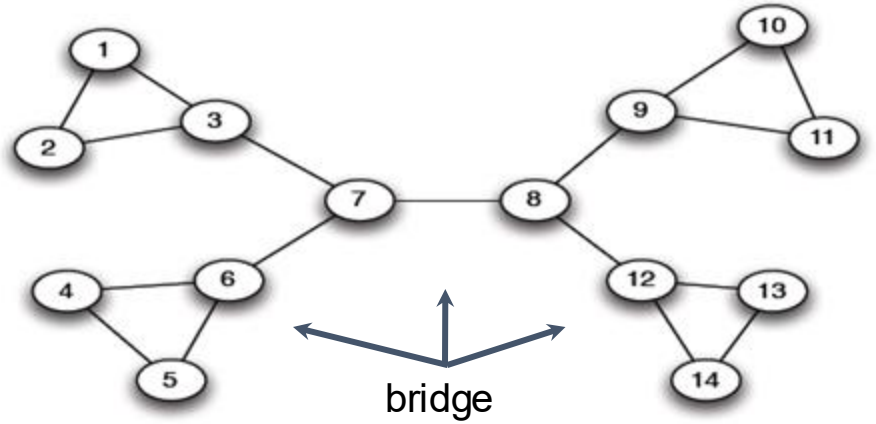
- We need a measure of how important an edge is in keeping the graph connected
- Edge betweenness: Number of shortest paths that pass through the edge

The edge betweenness centrality (EBC) can be defined as the number of shortest paths that pass through an edge (more specifically passing **through the endpoints of the edge**) in a network.

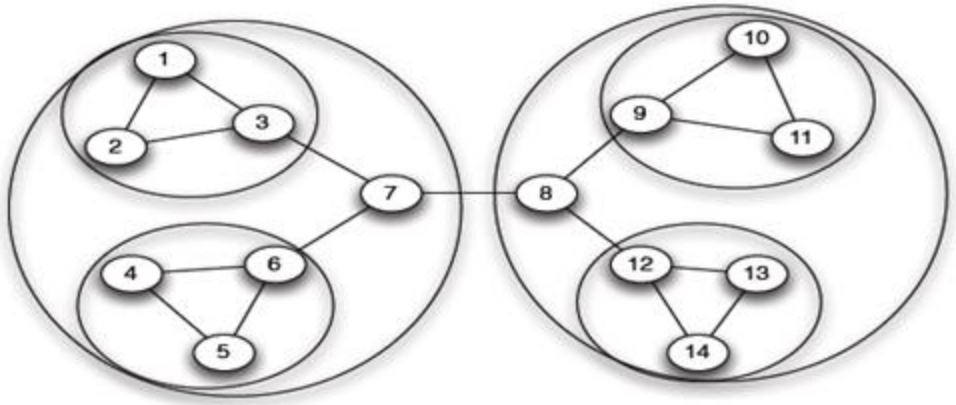


Betweenness of edge (*a*,*b*) ($B(a,b)$): For each pair of nodes x,y compute the number of shortest paths that include (a,b)

WHAT IS A COMMUNITY?



(a) *A sample network*



(b) *Tightly-knit regions and their nested structure*

Removal of a bridge separates the graph into disjoint components

An example of a nested structure of the communities

(Courtesy: Easley & Kleinberg)

Example: Practical Applications of Community Detection

Social Networks: Identifies individuals with similar interests; Helps understand social dynamics and group behavior.

Epidemiology: Recognizes clusters to prevent disease spread. Vital for public health strategies.

Internet Communities: Groups pages with similar content. Organizes online information effectively. Enables insights into diverse networks.

COMMUNITY DETECTION

Girvan-Newman Method [Newman and Girvan, 2004]

- Remove the edges of highest betweenness first.
- Repeat the same step with the remainder graph.
- Continue this until the graph breaks down into predefined or data-driven numbers of communities.

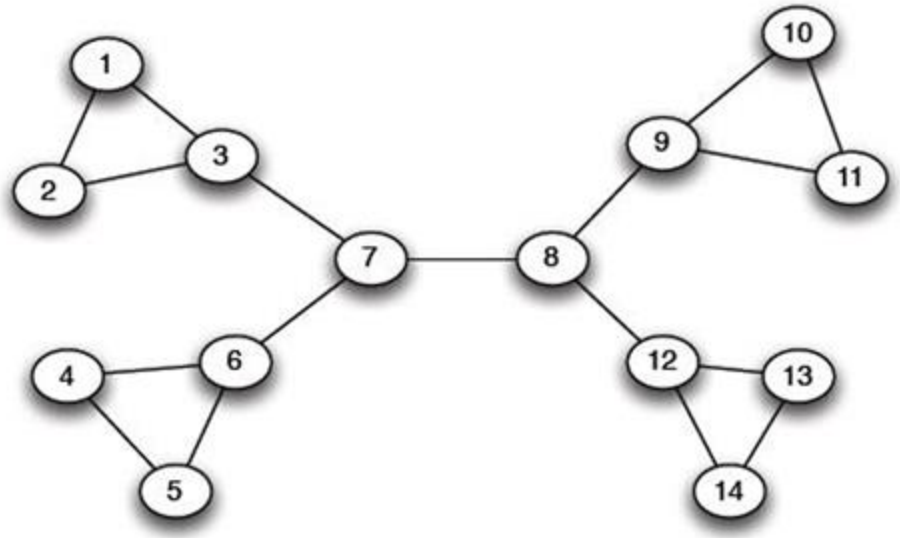
As the graph breaks down into pieces, the tightly knit community structure is exposed.

Results in a hierarchical partitioning of the graph (Gives a hierarchical decomposition of the graph into communities).

- So if we gradually remove the edge with the highest edge betweenness score we will get a hierarchical map, a rooted tree, called a dendrogram of the graph.
- The leaves of the tree are the individual vertices The root of the tree represents the whole graph

GIRVAN-NEWMAN METHOD

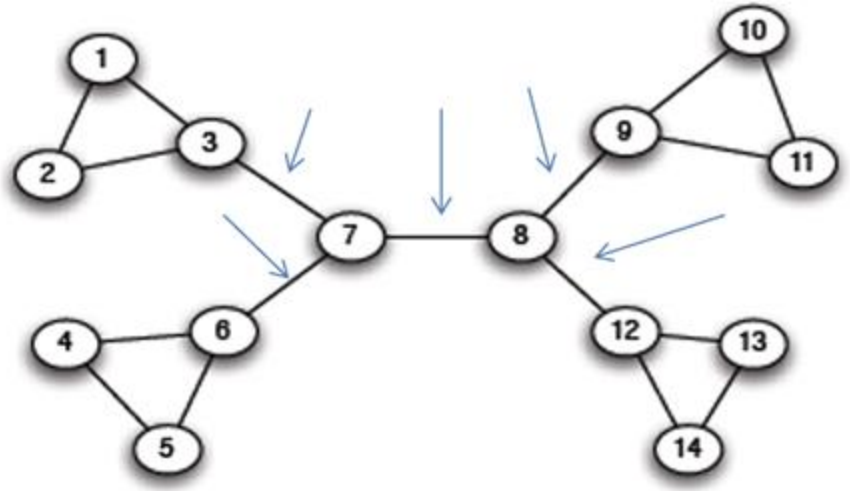
- Hierarchical divisive method
  - Start with the whole graph
  - Find edges whose removal "partitions" the graph
  - Repeat with each subgraph until single vertices



Which edge to remove?

GIRVAN-NEWMAN METHOD

- Select cut-edges (a.k.a. bridge edges): edges that when removed they disconnect the graph



- There may be many of those

The Girvan Newman Algorithm

- Given an undirected unweighted graph:
- Repeat until no edges are left:
  - Compute the edge betweeness for all edges
  - Remove the edge with the highest betweeness

- At each step of the algorithm, the connected components are the communities
- Gives a hierarchical decomposition of the graph into communities

Edge Betweeness

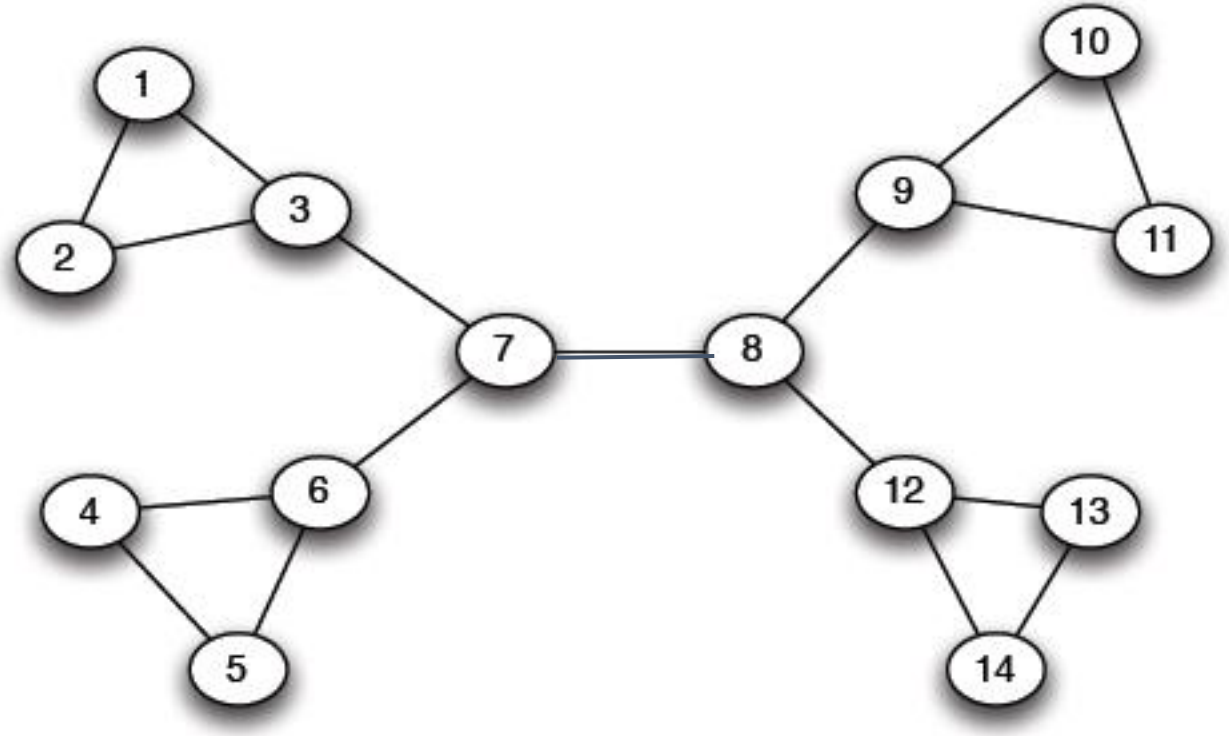
- Betweeness of edge (a, b) $(B(a, b))$:
  - For each pair of nodes x, y compute the number of shortest paths that include (a, b)
  - There may be multiple shortest paths between (x, y) $(SP(x, y))$. Compute the fraction of those that pass through (a, b)
    - Assumes a unit of traffic flow between (x, y)

$$B(a,b) = \sum_{x,y \in V} \frac{|SP(x,y) \; that \; include \; (a,b)|}{|SP(x,y)|}$$

- Betweenness computes the probability of an edge to occur on a randomly chosen shortest path between two randomly chosen nodes.

GIRVAN-NEWMAN METHOD

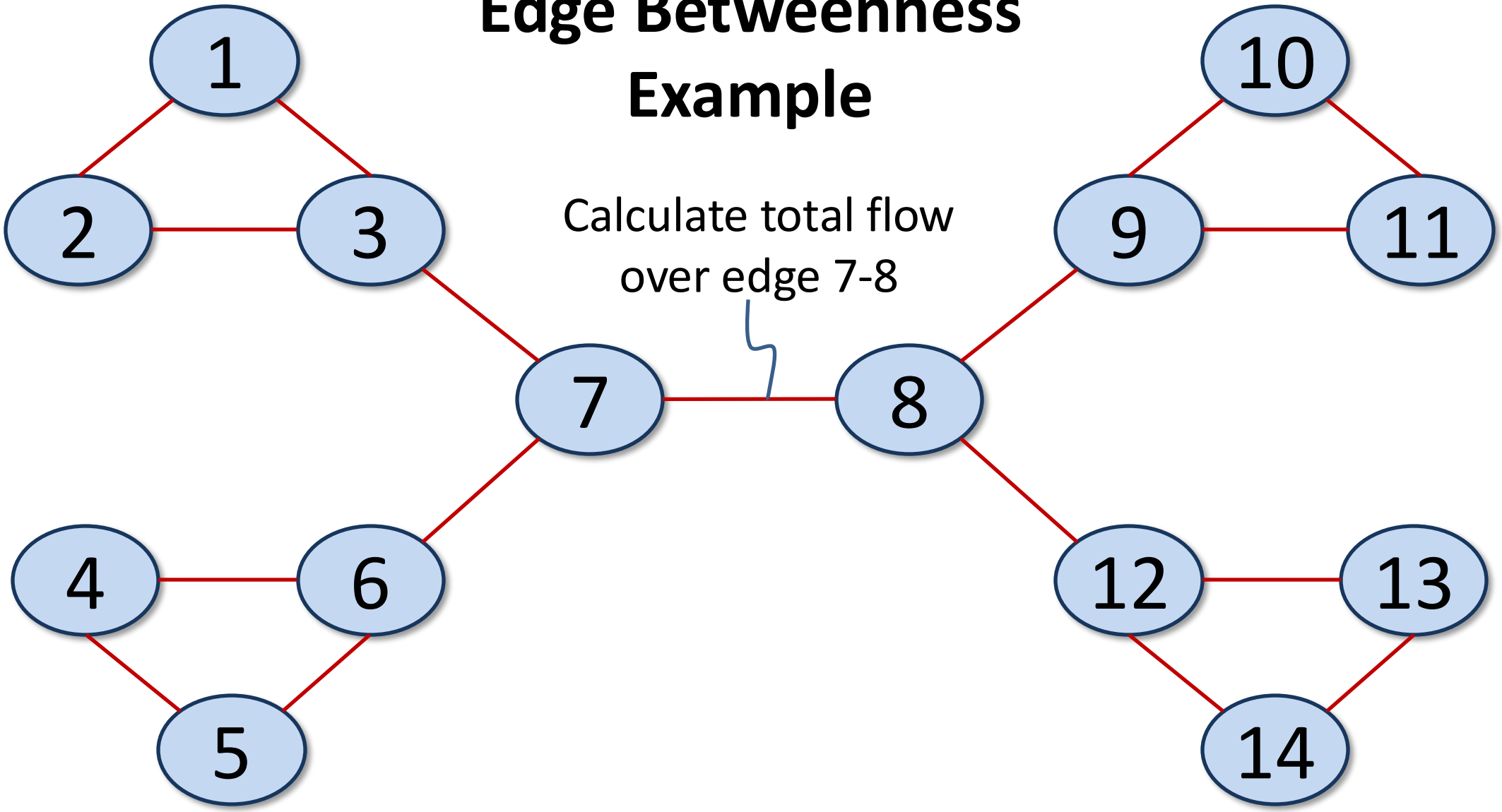
First, Calculate Edge Betweenness



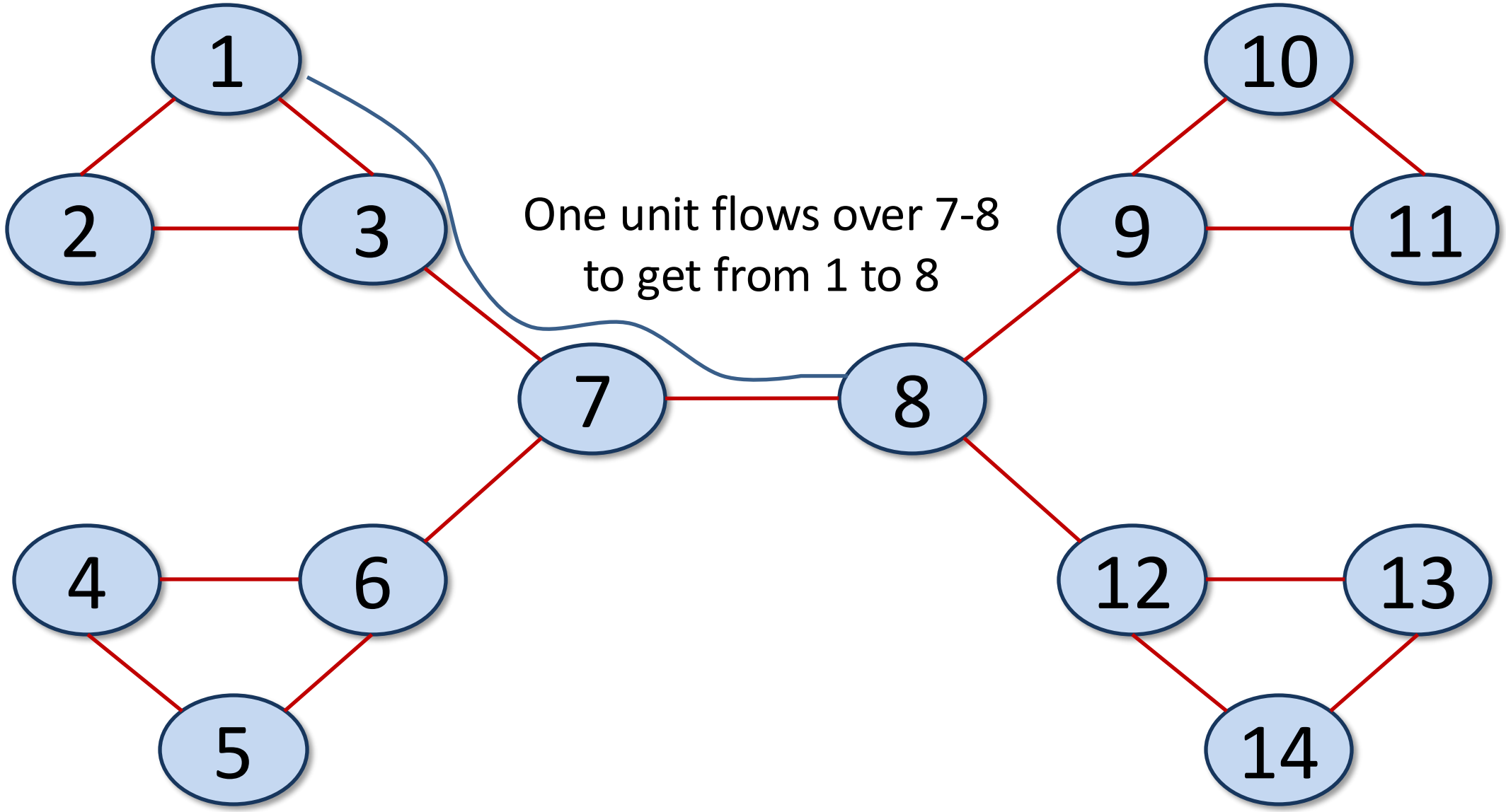
Betweenness(7-8)= 7*7 = 49

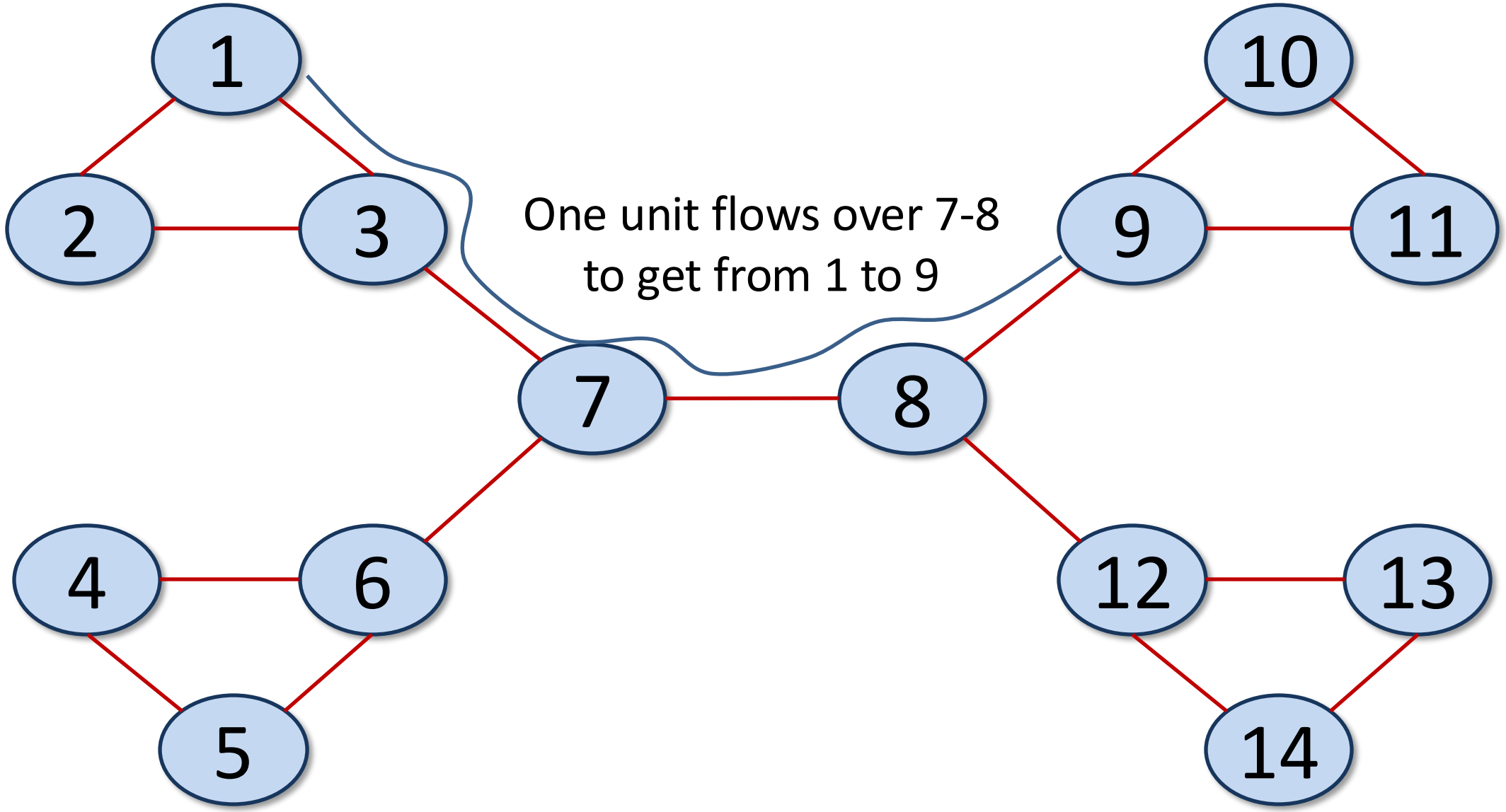
How?

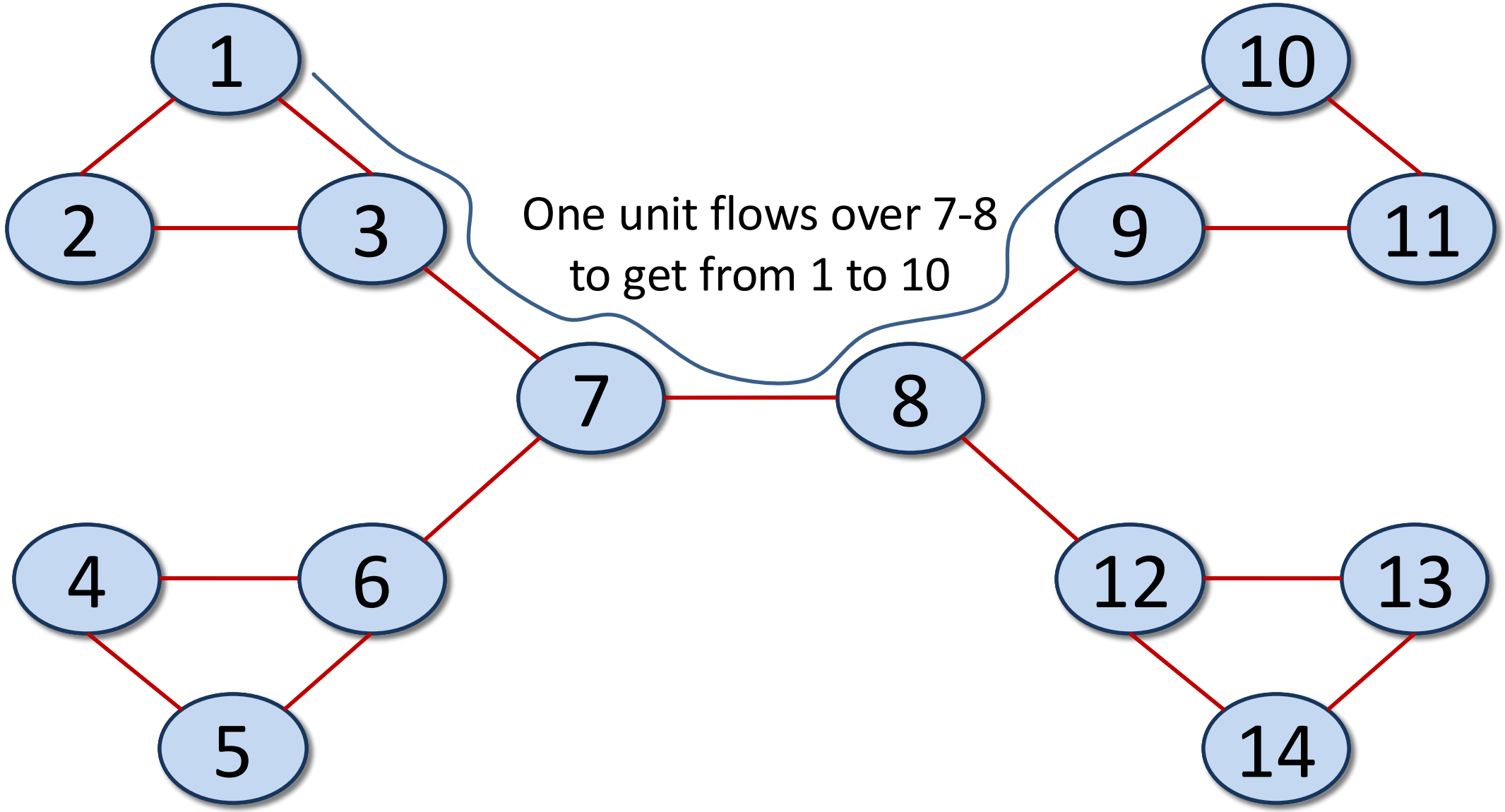
Edge Betweenness Example

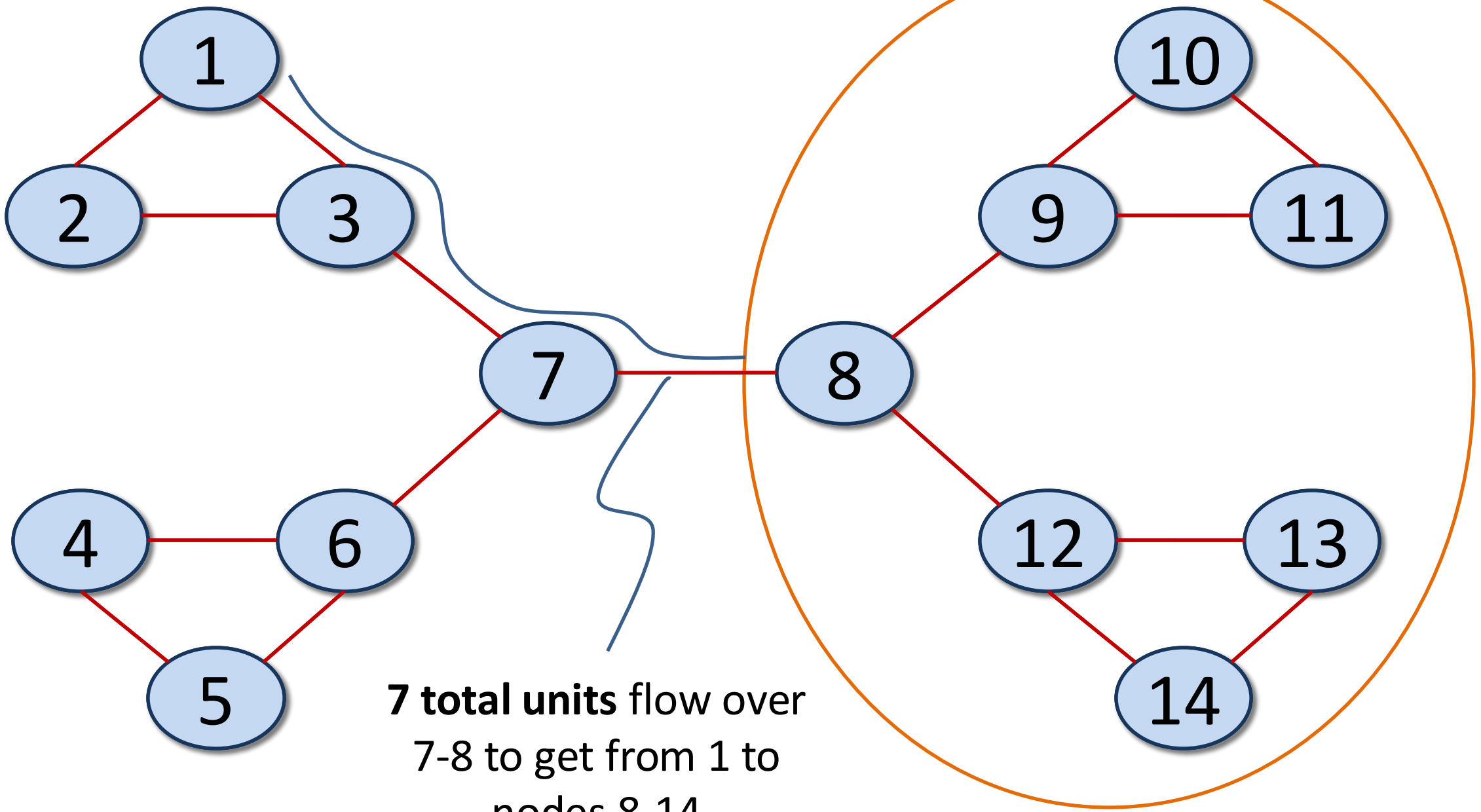


Edge 7-8 in the graph here carries info from each of the 7 nodes on the left (incl. node 7) and the 7 nodes on the right (incl. node 8) # shortest paths through Edge 7-8 is 7*7 = 49

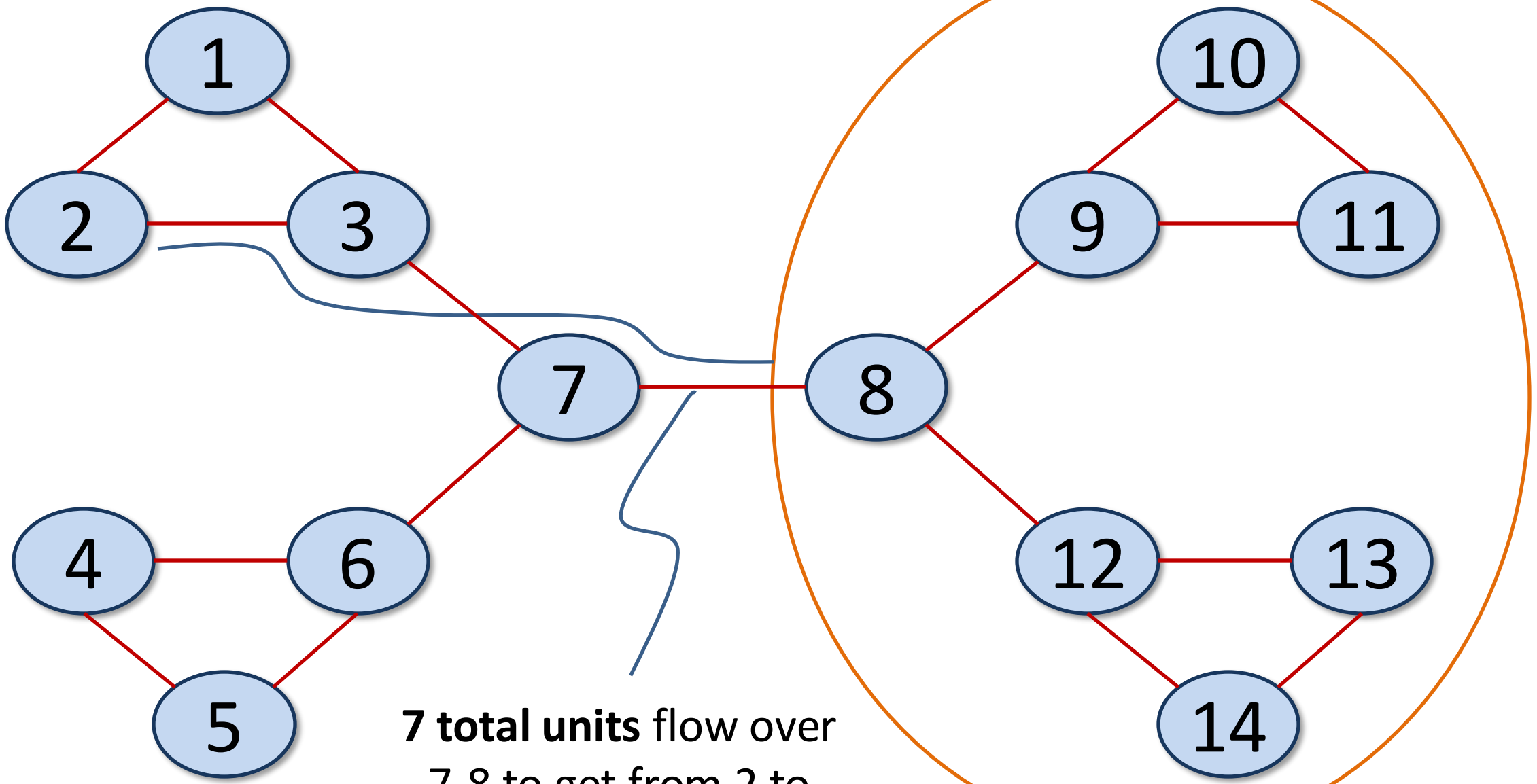

1
2
3
4
5
6
7
8
9
10
11
12
13
14
One unit flows over 7-8
to get from 1 to 8


1
2
3
4
5
6
7
8
9
10
11
12
13
14
One unit flows over 7-8
to get from 1 to 9

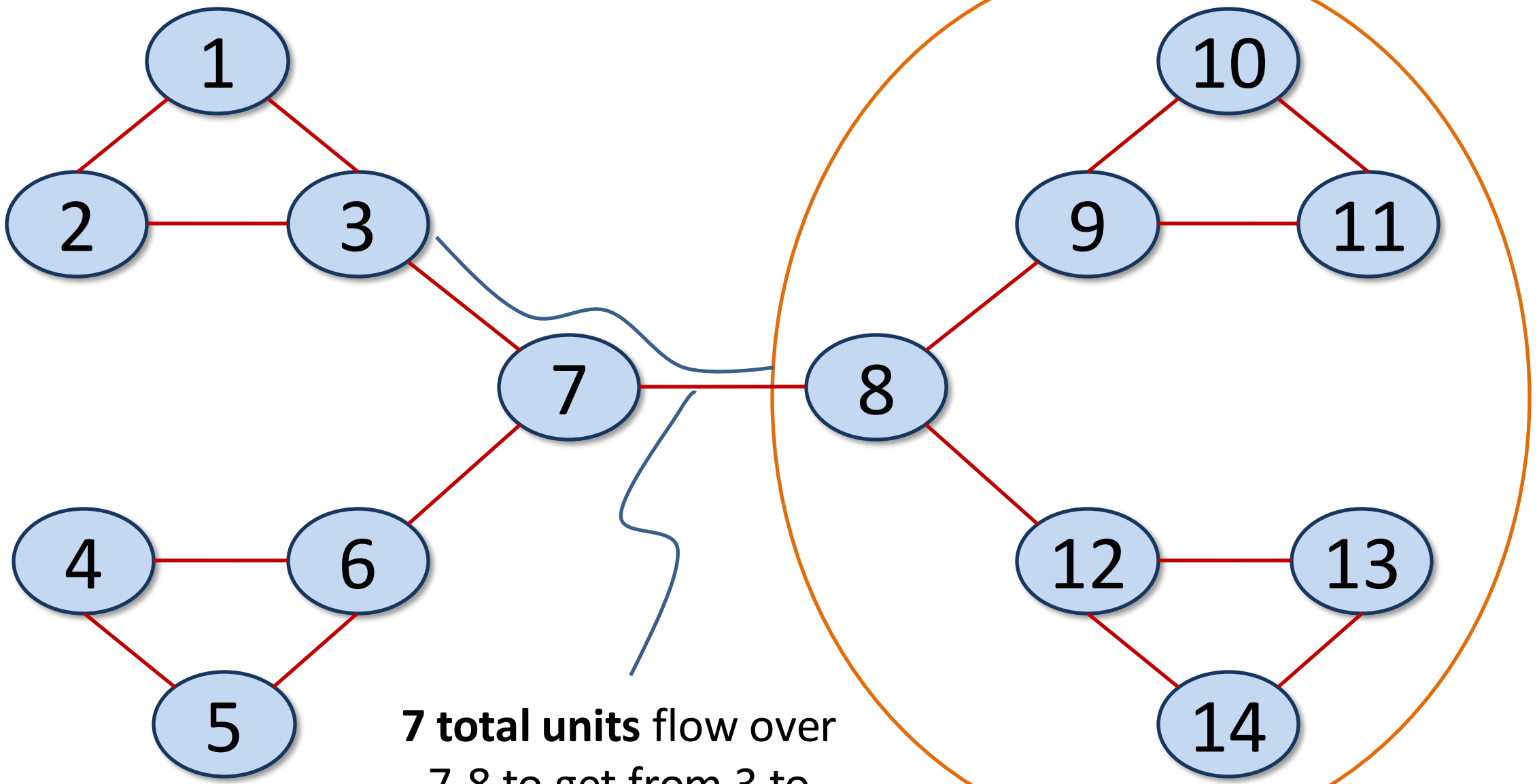

1
2
3
4
5
6
7
8
9
10
11
12
13
14
One unit flows over 7-8
to get from 1 to 10



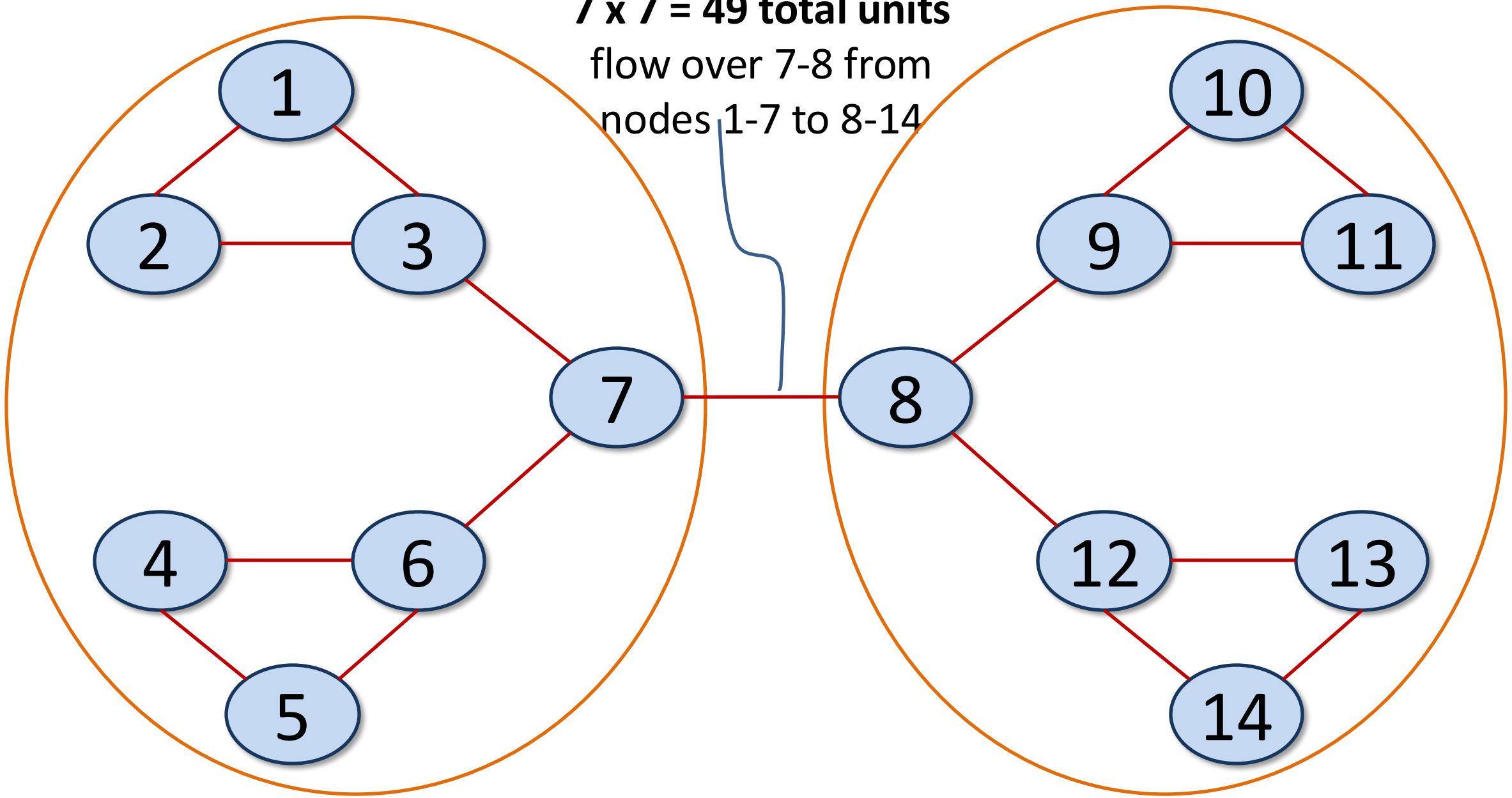
7 total units flow over 7-8 to get from 1 to nodes 8-14

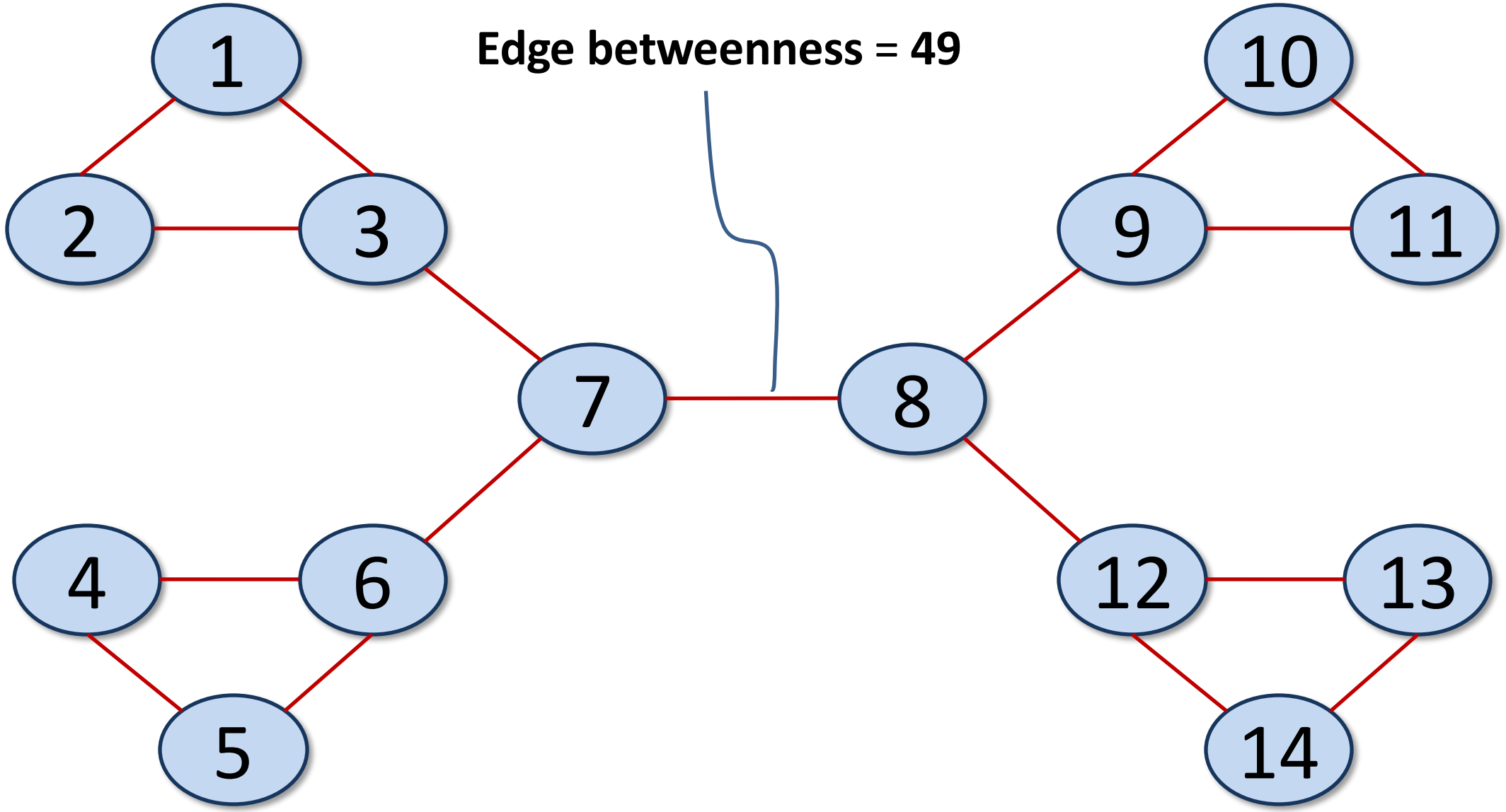


7 total units flow over 7-8 to get from 2 to nodes 8-14

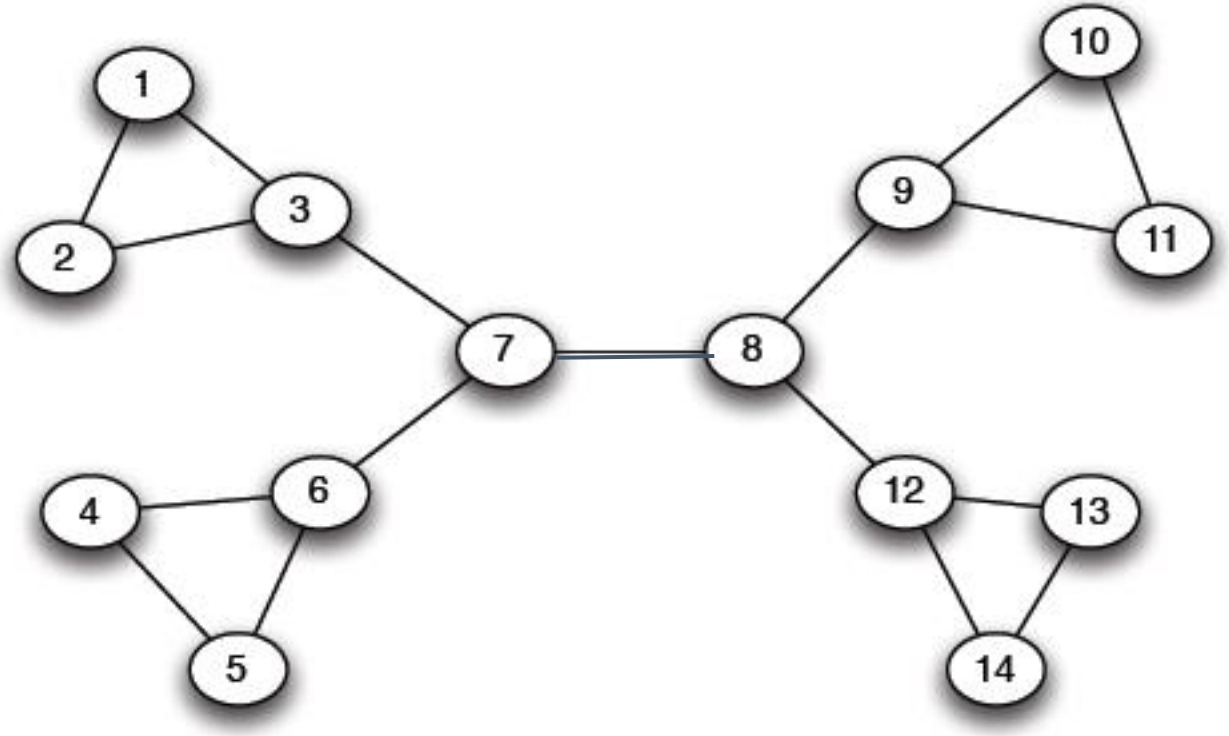


7 total units flow over 7-8 to get from 3 to nodes 8-14 and so on..


7 x 7 = 49 total units
flow over 7-8 from
nodes 1-7 to 8-14
1
2
3
4
5
6
7
8
9
10
11
12
13
14

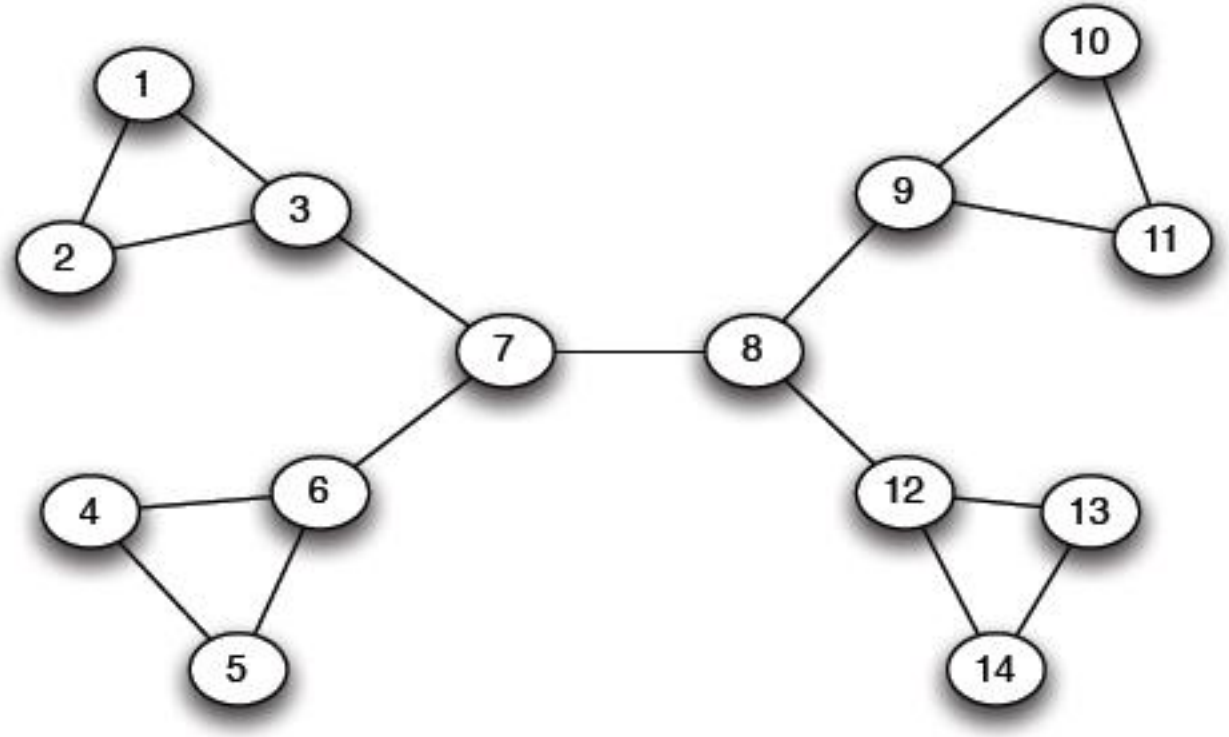

Edge betweenness = 49
1
2
3
4
5
6
7
8
9
10
11
12
13
14

Back to: GIRVAN-NEWMAN METHOD



Betweenness(7-8)= 7*7 = 49

GIRVAN-NEWMAN METHOD



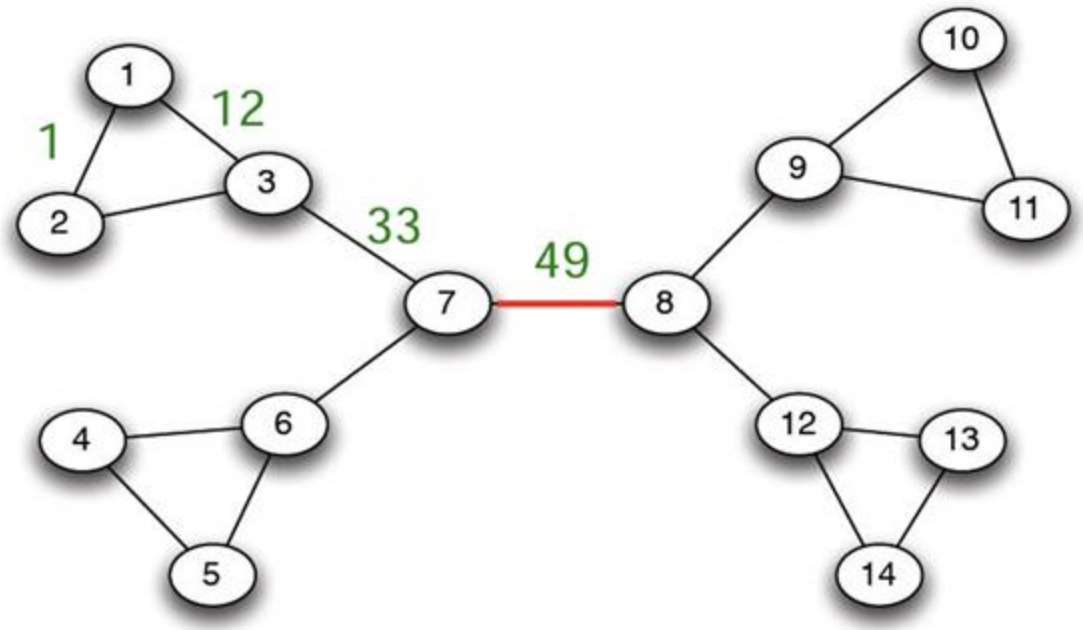
Betweenness(7-8)= 7*7 = **49**

Betweenness(1-3) = **?** **1*12 = 12**

Betweenness(3-7) = Betweenness(6-7) = Betweenness(8-9) = Betweenness(8-12) = **?** **3*11= 33**

Which edge is having highest betweenness?

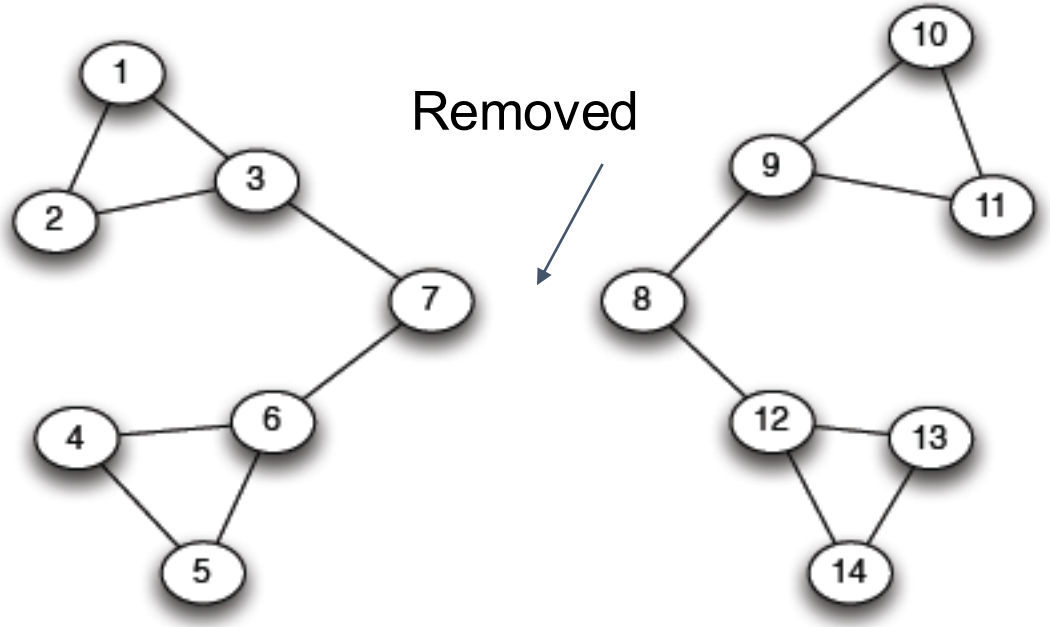
Girvan-Newman: Example



Need to re-compute betweenness at every step

GIRVAN-NEWMAN METHOD

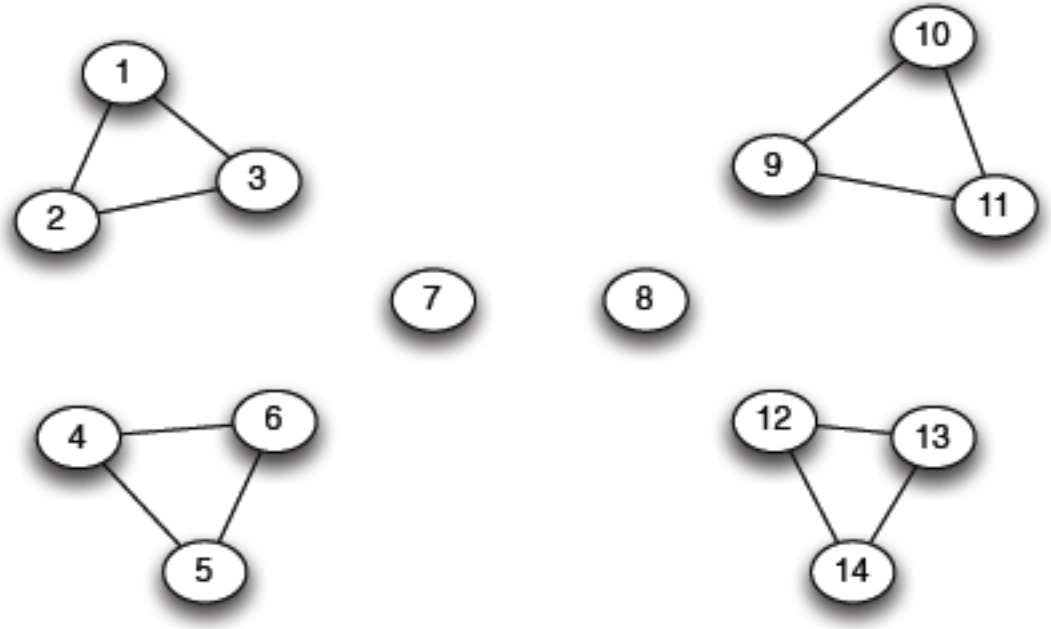
Next, Remove the edge having highest Edge Betweenness



(a) *Step 1*

Betweenness(1, 3) = 1X5=5
Betweenness(3,7) = Betweenness(6,7) = Betweenness(8,9) = Betweenness(8,12) = 3X4=**12**

GIRVAN-NEWMAN METHOD



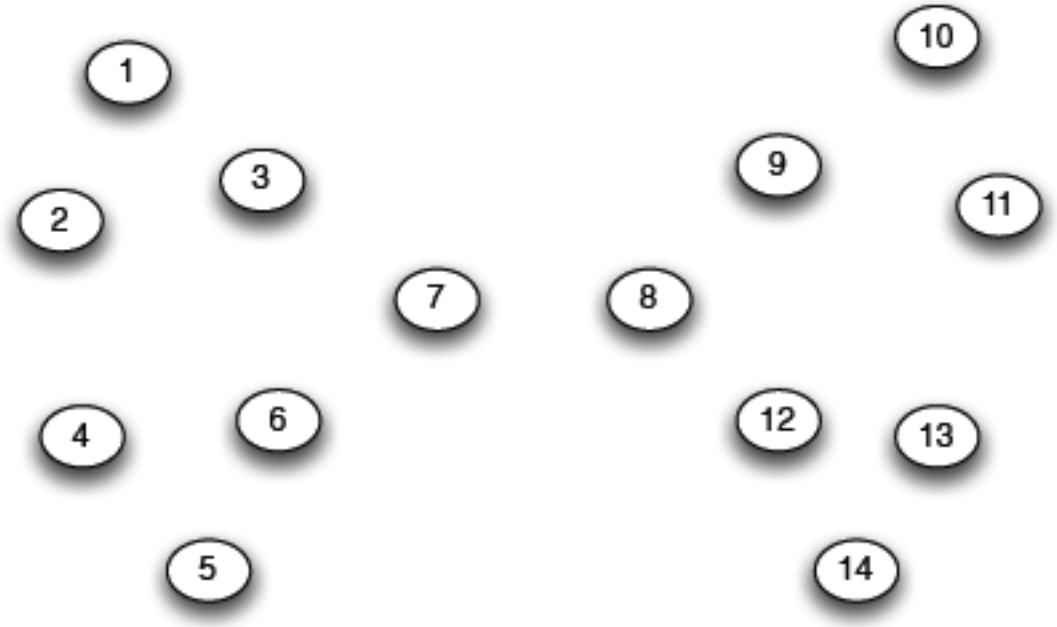
(b) *Step 2*

Repeat the Process

Why is Betweenness now??

Betweenness of every edge = 1

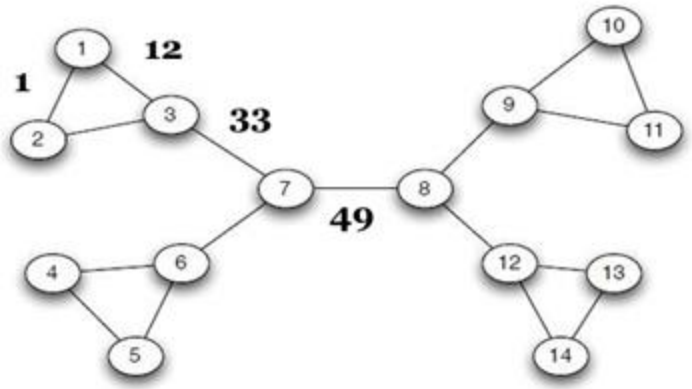
GIRVAN-NEWMAN METHOD

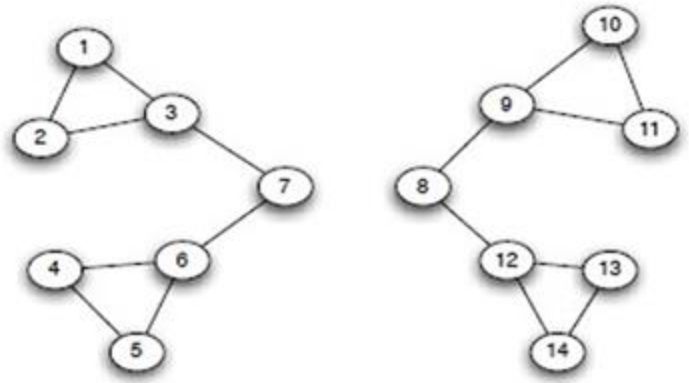



```
G=nx.Graph()

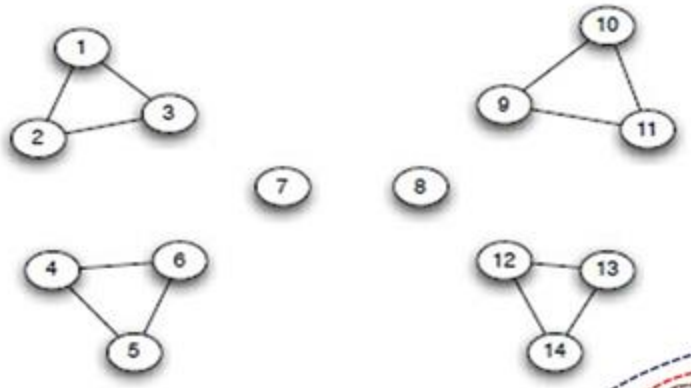
# Returns an iterator over partitions at
# different hierarchy levels
nx.girvan_newman(G)
```

ALL TOGETHER: GIRVAN-NEWMAN METHOD

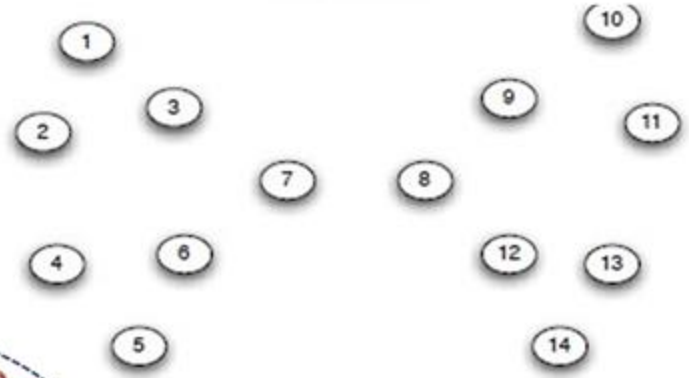




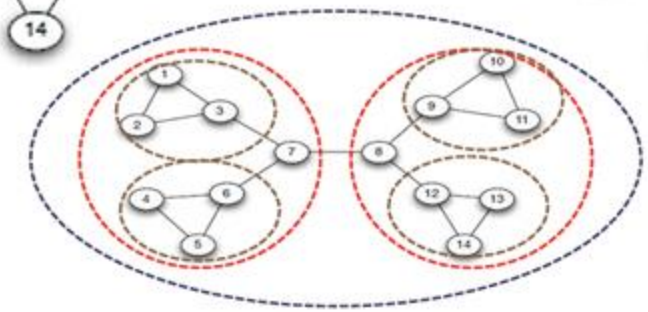
(a) *Step 1*



(b) *Step 2*



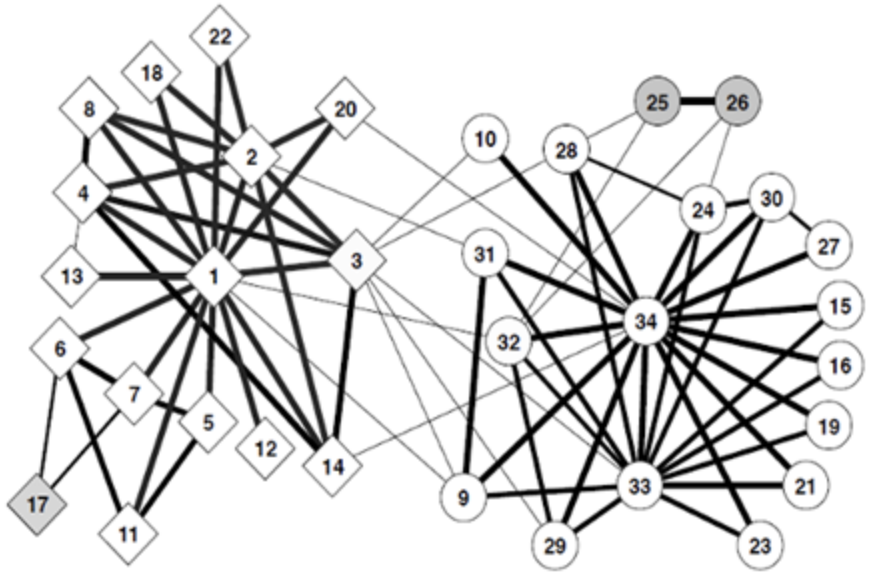
(c) *Step 3*

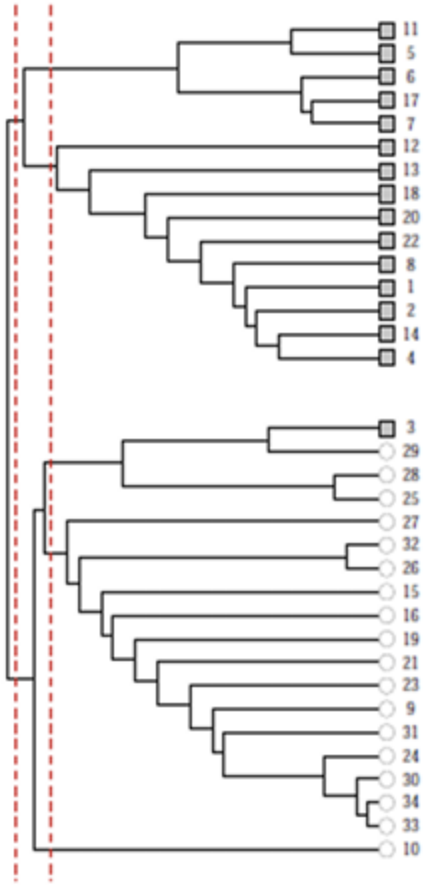


Communities are the resulting connected components!

A SAMPLE OF HOW HIERARCHICAL RESULTS OF GIRMAN-NEWMAN

- **Zachary's Karate club:**
  Hierarchical decomposition

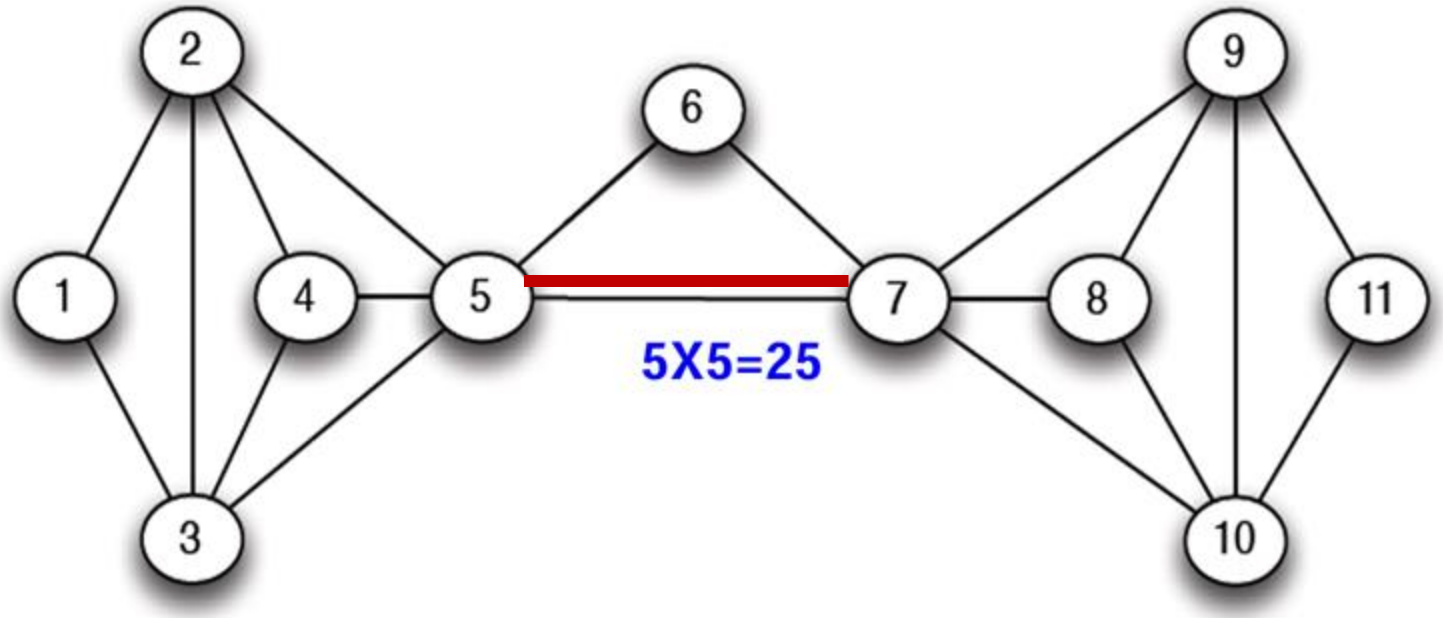




Results in a hierarchical partitioning of the graph.

- **So if we gradually remove the edge with the highest edge betweenness score we will get a hierarchical map, a rooted tree, called a dendrogram of the graph.**
- **The leaves of the tree are the individual vertices The root of the tree represents the whole graph**

Another example



GIRVAN-NEWMAN METHOD

The method gives us only a succession of splits of the network into smaller and smaller communities, but it gives no indication of which splits are best.

- There can be a problem of “Over splitting”

One way to find the best split is via the the **modularity concept**.

- Modularity is a **measure of the strength of division of a network into modules or communities.**

An Improvement: Introducing Modularity

“Modularity” - a helpful metric to assess the strength of the community structure found.

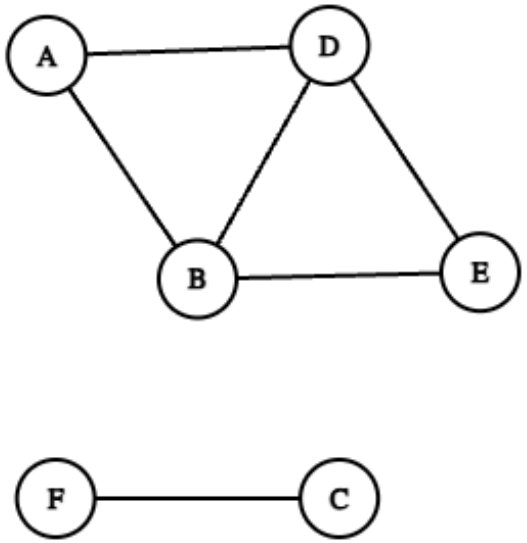
- We can use Modularity to evaluate community structures.

Modularity can be represented and calculated in different ways depending on the specific community detection algorithm. We will focus on “Modularity Score Q”.

Modularity

Modularity quantifies the difference between the number of edges within modules and the expected number of edges if the edges were distributed randomly.

A higher modularity value indicates a better community structure, where there are more edges within modules and fewer between them.



Modularity

After removing an edge, the Girvan-Newman algorithm can be used to calculate the modularity (Q) of the graph.

A higher value suggests a more significant community structure.

modularity values typically range from -1 to 1, with values closer to 1 indicating a strong division. In practice, the Girvan-Newman algorithm would iteratively remove edges and recalculate modularity to find a division that maximizes Q.

```
float oldModularity = 0.0001, newModularity;

// 1. check the modularity
while (newModularity > oldModularity) {

    // 2. Find edge with the highest betweenness
    pair<int,int> edge = algo.topEdge();

    // 3. Remove the edge
    graph.removeEdge(edge.first, edge.second);

    // 4. recalculate the modularity of updated graph
    newModularity = algo.getModularity();

}
```

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

Where:

- $A_{ij} = 1$ if there is an edge between node *i* and *j*, 0 otherwise
- k_i = degree of node *i*
- m = total number of edges in the graph
- c_i = community of node *i*
- $\delta(c_i, c_j) = 1$ if nodes *i* and *j* are in the same community, 0 otherwise

Suppose you have a network of 4 nodes and 4 edges:

- **Nodes:** A, B, C, D
- **Edges:** (A-B), (B-C), (C-D), (D-A)

This forms a **square (cycle)**. Now, you divide the nodes into two communities:

- **Community 1:** A, B
- **Community 2:** C, D

Step 1: Adjacency Matrix A

We assign indices to the nodes for matrix representation:
A → 0, B → 1, C → 2, D → 3

Then the adjacency matrix A becomes:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

Step 2: Degrees k_i

Each node has degree 2:

- $k_0 = k_1 = k_2 = k_3 = 2$

Step 3: Total number of edges m

Each edge appears twice in the adjacency matrix, so:

$$m = \frac{1}{2} \sum_i k_i = \frac{1}{2}(2 + 2 + 2 + 2) = 4$$

Step 3: Total number of edges m

Each edge appears twice in the adjacency matrix, so:

$$m = \frac{1}{2} \sum_i k_i = \frac{1}{2}(2 + 2 + 2 + 2) = 4$$

Step 4: Communities

Let's define community labels:

- $c_0 = c_1 = 1$ (nodes A and B)
- $c_2 = c_3 = 2$ (nodes C and D)

Step 5: Modularity Formula

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

We sum only over **pairs in the same community**:

Community 1 (A, B): i,j ∈ {0,1}

- $(0, 0)$: $A_{00} = 0$, $\frac{2 \cdot 2}{8} = 0.5$, contribution = $0 - 0.5 = -0.5$
- $(0, 1)$: $A_{01} = 1$, contribution = $1 - 0.5 = 0.5$
- $(1, 0)$: same as (0,1) = 0.5
- $(1, 1)$: $A_{11} = 0$, contribution = $0 - 0.5 = -0.5$

Total from Community 1:

$$-0.5 + 0.5 + 0.5 - 0.5 = 0$$

Community 2 (C, D): i,j ∈ {2,3}

- $(2, 2)$: $A_{22} = 0$, contribution = -0.5
- $(2, 3)$: $A_{23} = 1$, contribution = 0.5
- $(3, 2)$: same = 0.5
- $(3, 3)$: $A_{33} = 0$, contribution = -0.5

Total from Community 2:

$$-0.5 + 0.5 + 0.5 - 0.5 = 0$$

Final Modularity:

$$Q = \frac{1}{2 \cdot 4} \cdot 0 = 0$$

A modularity of **0** suggests that this particular division into two communities **does not reflect a strong community structure**

NETWORKX: VIZ

Can render via Matplotlib or GraphViz

```
import matplotlib.pyplot as plt

G=nx.Graph()
nx.draw(G, with_labels=True)

# Save to a PDF
plt.savefig("my_filename.pdf")
```

Many different layout engines, aesthetic options, etc

- https://networkx.github.io/documentation/networkx-1.10/reference/drawing.html
- https://networkx.github.io/documentation/development/gallery.html

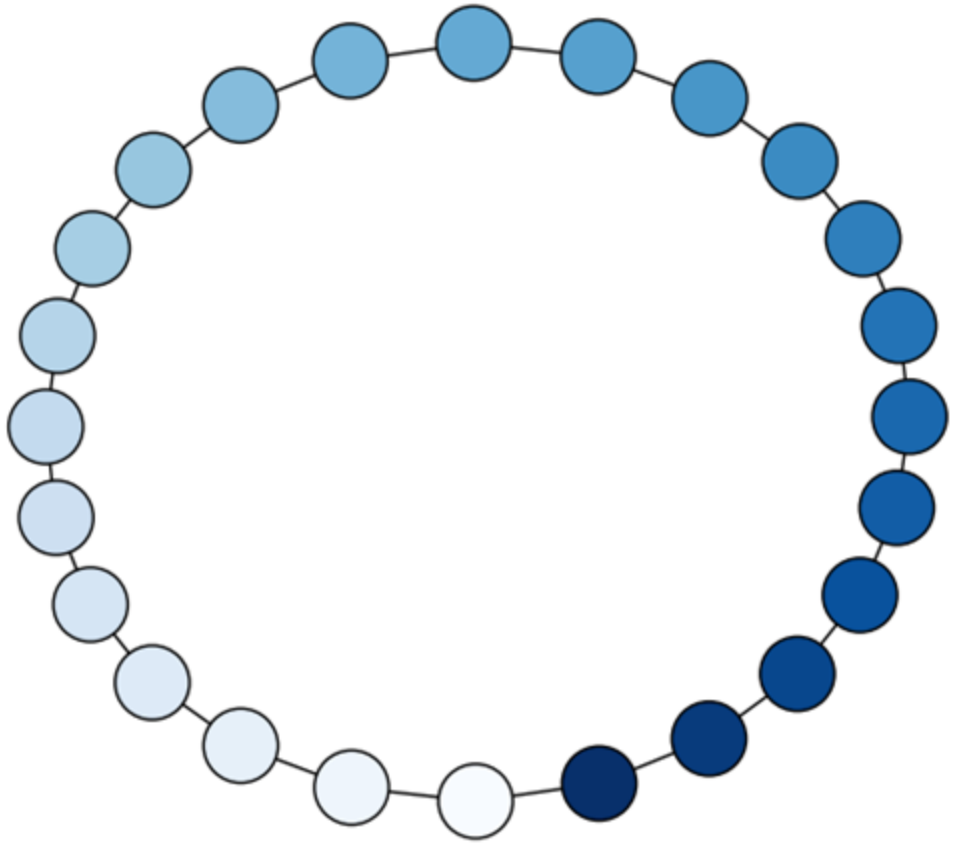
NETWORKX: VIZ

```
# Cycle with 24 vertices
G=nx.cycle_graph(24)

# Compute force-based layout
pos=nx.spring_layout(G,
          iterations=200)

# Draw the graph
nx.draw(G,pos,
      node_color=range(24),
      node_size=800,
      cmap=plt.cm.Blues)

# Save as PNG, then display
plt.savefig(”graph.png")
plt.show()
```



NETWORKX: VIZ

```
# Branch factor 3, depth 5
G = nx.balanced_tree(3, 5)

# Circular layout
pos = graphviz_layout(G,
      prog='twopi', args='')

# Draw 8x8 figure
plt.figure(figsize=(8, 8))
nx.draw(G, pos,
          node_size=20,
          alpha=0.5,
          node_color="blue",
          with_labels=False)

plt.axis('equal')
plt.show()
```

